# Automated Construction of Semantic Knowledge Graphs from Structured Data

**Submitted to the**
**Albert-Ludwigs-Universität Freiburg**

**by Surbhi Nair**




universität freiburg

# Master's Thesis

Submitted on **October 26, 2026**
to the **Albert-Ludwigs-Universität Freiburg**
by **Surbhi Nair** (5973324)

Examined by
**Prof. Dr. Hannah Bast**
**Prof. Dr. Joschka Bödecker**

Department of Computer Science
Faculty of Engineering
Albert-Ludwigs-Universität Freiburg

# Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used.

Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg, October 26, 2026

| | |
|---|---|
| Place, date | Surbhi Nair |

# Statement on AI Usage

# Acknowledgements

# Abstract

# Contents


**1 Introduction** **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2

**2 Related Work** **7**
- 2.1 Conventional Approaches . . . 7
  - 2.1.1 Direct Mapping . . . 7
  - 2.1.2 Declarative Mappings . . . 7
- 2.2 LLM-Based Knowledge Graph Construction . . . 8
  - 2.2.1 Direct Triple Extraction . . . 8
  - 2.2.2 Mapping Generation . . . 9
- 2.3 Competency-Question-Driven Modeling . . . 10
- 2.4 Evaluation in Related Work . . . 12

**3 Background** **13**
- 3.1 SPARQL CONSTRUCT Queries . . . 13
- 3.2 LLM Agents and Tool Calling . . . 16
- 3.3 GRASP . . . 16

**4 Approach** **18**
- 4.1 Structured Data to Generic KG . . . 19
- 4.2 Preparing the Generic KG . . . 22
- 4.3 Generating CONSTRUCT Queries . . . 22
- 4.4 Baseline . . . 29

**5 Evaluation** **31**
- 5.1 Datasets . . . 31
  - 5.1.1 BIRD Databases . . . 31
  - 5.1.2 Mondial . . . 33
  - 5.1.3 IMDb . . . 34
  - 5.1.4 MusicBrainz . . . 35
- 5.2 Setup . . . 36
- 5.3 Results . . . 39
  - 5.3.1 Syntactic Validity . . . 39
  - 5.3.2 Judge Scores . . . 41
  - 5.3.3 Qualitative Analysis . . . 44

**6 Conclusion** **48**

**List of Figures** **I**

**List of Tables** **II**

**Bibliography** **III**

# 1 Introduction

A large share of today's data is stored in relational databases and CSV files. Converting such data into RDF knowledge graphs (KGs) makes it reusable and queryable beyond the application it was designed for, but only if the data is modeled well. In this thesis, we investigate to what extent an LLM agent can automate this modeling by generating SPARQL CONSTRUCT queries.

## 1.1 Motivation

The quality of converting structured data, such as relational databases or CSV files, into an RDF knowledge graph (KG) depends on how the data is modeled. Making structured data queryable with SPARQL is straightforward: the direct mapping from relational data to RDF has been standardized by the W3C [2]. It turns every row into a node and every column into a predicate so the resulting graph reflects how the data is stored rather than what it describes. A semantically modeled KG, in contrast, represents domain concepts as entities, describes them with meaningful classes and properties, and expresses relationships between them using a well-designed ontology.

Building a semantically modeled KG like this requires substantial manual effort and expertise: the ontology is traditionally designed manually with collaboration between domain experts and ontology specialists, often following methodologies that range from classical approaches [11] to agile, industry-oriented frameworks such as LOT [25].

Tools have automated the mechanical parts of the conversion: mapping languages such as RML [10] declare how source data maps to RDF, engines such as Morph-KGC [3] execute these mappings at scale, and Tarql [8] transforms CSV files with hand-written SPARQL CONSTRUCT queries. The central part, however, remains manual: understanding the domain and the source schema, and deciding how the data should be modeled. At scale, this becomes all the more challenging.

Large language models (LLMs) are well-suited for this task. The required modeling decisions depend largely on the meaning of table names, column names, and values, which LLMs can interpret. Recent work confirms that LLMs can indeed be used to convert structured data into KGs.

Existing approaches follow two directions: direct triples extraction by an LLM [14, 24], which is severely limited by the size of the data, or the LLM generates a declarative mapping, for example in RML, which a mapping engine then executes [16, 26]. These

experiments, however, are small in scale and domain-specific, or they rely on a target ontology that is given in advance.

One direction that to our knowledge has not been explored yet, is to let an LLM generate SPARQL CONSTRUCT queries that transform an input KG into a semantically modeled one. This direction combines the advantages of the other two. Like a mapping, a CONSTRUCT query is an explicit artifact, which a human can inspect and correct, and its size depends only on the schema, not on the number of rows. Unlike an RML mapping, it requires no separate engine, since the query can be executed by the same SPARQL engine that hosts the data.

In this thesis, we therefore investigate to what extent an LLM agent can automate the semantic modeling of structured data by generating SPARQL CONSTRUCT queries, for relational datasets of varying schema complexity and without a target ontology specifically provided.

## 1.2 Problem Statement

The goal of this thesis is to automate the step from the output of a direct mapping or a pre-existing knowledge graph to a semantically modeled one. Before we define this task formally, we illustrate the difference with a simple example.

Figure 1 shows three tables of the IMDb dataset. The table `principals` connects titles to persons and records the role of each person in the column `category`.

**title**

| tconst | titleType | primaryTitle |
|---|---|---|
| tt1 | movie | Heat |

**person**

| nconst | primaryName |
|---|---|
| nm01 | Michael Mann |
| nm02 | Al Pacino |

**principals**

| tconst | ordering | nconst | category |
|---|---|---|---|
| tt1 | 1 | nm01 | director |
| tt1 | 2 | nm02 | actor |

**Figure 1:** Three tables of the IMDb dataset with selected columns.

In the generic KG shown in Figure 2, every row of `principals` becomes a node of its own, which sits between the movie and the person, and the role of the person is only a string attached to this node instead of being expressed as a predicate. The semantic KG instead connects the movie directly to its director and its actor, with the role expressed by the predicates `imdb:director` and `imdb:actor`. A direct mapping cannot produce such a model, because it would have to derive predicates from the values of a column.

```
<row/title/tt1> a <table/title> ;
  <title/titleType>     "movie" ;
  <title/primaryTitle>  "Heat" .

<row/person/nm01> a <table/person> ;
  <person/primaryName>  "Michael Mann" .
<row/person/nm02> a <table/person> ;
  <person/primaryName>  "Al Pacino" .

<row/principals/tconst=tt1;ordering=1> a <table/principals> ;
  <principals/category>   "director" ;
  <ref/principals/tconst> <row/title/tt1> ;
  <ref/principals/nconst> <row/person/nm01> .

<row/principals/tconst=tt1;ordering=2> a <table/principals> ;
  <principals/category>   "actor" ;
  <ref/principals/tconst> <row/title/tt1> ;
  <ref/principals/nconst> <row/person/nm02> .
```

```
title:tt1 a imdb:Movie ;
  rdfs:label    "Heat" ;
  imdb:director name:nm01 ;
  imdb:actor    name:nm02 .
name:nm01 a imdb:Person ; rdfs:label "Michael Mann" .
name:nm02 a imdb:Person ; rdfs:label "Al Pacino" .
```

**Figure 2:** The generic KG of the IMDb tables (top) and a semantic KG of the same data (bottom).

The difference becomes visible when the data is queried. To ask who directed *Heat*, a query over the generic KG has to traverse the intermediate node and filter it by the string `"director"`, while a query over the semantic KG follows a single edge.

```
SELECT ?name WHERE {
  ?t <title/primaryTitle> "Heat" .
  ?p <ref/principals/tconst> ?t ;
     <principals/category> "director" ;
     <ref/principals/nconst> ?d .
  ?d <person/primaryName> ?name .
}
```

```
SELECT ?name WHERE {
  ?t a imdb:Movie ;
     rdfs:label "Heat" ;
     imdb:director ?d .
  ?d rdfs:label ?name .
}
```

**Figure 3:** The query “Who directed *Heat*?” over the generic KG (bottom left) and over the semantic KG (bottom right).

The semantic KG shown in this example is, however, only one of many valid choices. Another model could keep each row of `principals` as a node, but as an entity of the domain: a *credit* of class `imdb:Credit`, which links the movie and the person, and refers to the role as an entity, such as `imdb:Director`, rather than as a string. Unlike the direct edges in Figure 3, this model also preserves the billing order recorded in the column `ordering`, at the cost of slightly longer queries. Both models are reasonable even while they differ slightly in structure. Since no target ontology is given, there is thus no single correct semantic KG, and therefore no reference graph against which a result could be compared. Instead, the quality of a semantic KG has to be judged by criteria that are independent of the chosen vocabulary and structure.

To formalise this problem statement, we first introduce the key terminology used.

- $D$ is the input dataset, which has to be converted into a semantic KG. In this thesis, $D$ is structured data, that is, a relational database or a set of CSV or TSV files.

- $G_D$ is the *generic KG* of $D$: the RDF graph obtained from $D$ by a direct mapping in the style of the W3C Direct Mapping [2]. Section 4 describes this in detail.

- $C = \{c_1, \ldots, c_m\}$ is an optional set of competency questions: natural-language questions that the semantic KG should be able to answer [13].

- $Q = \{q_1, \ldots, q_k\}$ is a set of SPARQL CONSTRUCT queries. Executing $Q$ on $G_D$ yields the graph
$$Q(G_D) = \bigcup_{i=1}^{k} q_i(G_D).$$

- $G = Q(G_D)$ is the output KG.

**Task:** Given the generic KG $G_D$ derived from $D$ and, optionally the competency questions $C$, the task is to find a set of CONSTRUCT queries $Q$ such that $G = Q(G_D)$ is a semantic KG of $D$. Choosing the classes and predicates of $G$ is part of the task. We judge whether $G$ is a good semantic KG of $D$ by three main requirements:

1. $G$ *models the domain* of $D$, not the structure of its schema;

2. $G$ is *faithful* to $D$: it preserves the information of $D$ and adds nothing that $D$ does not claim; and

3. $G$ is *usable*: questions about the data can be answered over $G$ with simple queries.

**Criteria:** We make each requirement concrete with a set of criteria.

**Semantic Modeling:**

- *Entity and class modeling:* It concerns selection and granularity. Which real-world entities or domain concepts from the source data become entities in the semantic KG? Are the classes and entities at the right level of granularity, or are they too coarse or too fine?

- *Vocabulary:* Do the classes and predicates of $G$ denote concepts of the domain, instead of repeating the source table and column names? And does each predicate have a single, precise meaning and the appropriate or preferred direction?

- *IRI design:* Are entity IRIs minted from stable identifiers of the source data, in a namespace that reflects the domain, and does the same entity receive the same IRI wherever it is referenced?

- *Relationships:* Where reference information is available, are the relationships of the source domain recoverable from $G$, without $G$ being a structural replica of the source schema?

**Fidelity:**

- *Schema coverage:* Is every data element(required tables/columns) of $D$ carried over into $G$?

- *Soundness:* Is everything in $G$ supported by $D$, that is, does $Q$ not invent entities, values, or links?

- *Structural integrity:* Is $G$ internally consistent? Every link should point to an entity that is actually described in $G$, and each real-world entity should appear as exactly one node. For example, the same person should not appear under two different IRIs because two queries mint its IRI differently.

- *Literal typing:* Do literals carry datatypes that match their values?

**Usability:**

- *Answerability:* Can a natural language question be answered over $G$ with a SPARQL query, does this query return the correct answer, and how complex is it compared to a query over $G_D$?

**Research questions:** Based on this task and these criteria, overall we aim to address the following questions from our experiments:

**RQ1.** Can an LLM agent that explores $G_D$ generate a set of CONSTRUCT queries $Q$ such that $Q(G_D)$ can be considered a good semantic model as per the criteria above?

**RQ2.** Does the approach proposed in this work yield better semantic modeling than a general-purpose LLM agent that is given the same input KG?

**RQ3.** Does supplying competency questions as additional input change the resulting modeling, and does it improve it?

**RQ4.** How does modeling quality change as the size and relational complexity of the schema grow, and what are the characteristic failure modes?

**Scope:** The input is structured data, that is, relational databases and CSV/TSV files, and not unstructured data such as text. Each dataset comes from a single source; we do not integrate several sources into one KG. The output is a materialized RDF graph, not a virtual graph over the source data and not a labeled property graph. The agent may treat placeholder values as missing, but correcting errors in the source data is not a part of the task. For large datasets, the agent explores a sample of the data rather than the complete data. Section 4 describes this implementation in detail.

# 2 Related Work

This chapter reviews the related work done on KG construction or conversion of structured data into RDF knowledge graphs starting from conventional approaches to more recent work that uses LLMs.

## 2.1 Conventional Approaches

### 2.1.1 Direct Mapping

The simplest conversion of structured data into RDF is a direct mapping. The W3C Direct Mapping [2] defines how a relational database can be converted into RDF without any user input: every table yields a class, every row a subject, and every column a predicate; foreign keys yield links between rows.

$$\text{Source Data}(D) \xrightarrow{\text{direct mapping}} \text{generic KG } (G_D)$$

$$\text{Direct Mapping} = \{\text{Table} \rightarrow \text{Class}, \text{Row} \rightarrow \text{Subject}, \text{Column} \rightarrow \text{Predicate}\}$$

Direct mappings are fully automatic, one can use simple scripts to perform such a conversion, but their output reflects the structure of the source schema rather than the domain it describes. Their main aim is simply to make the data queryable through SPARQL. It does not provide a semantic interpretation of the data. In this thesis, we use a simplified direct mapping only as the first step, to obtain the generic KG $G_D$ (Section 1.2).

### 2.1.2 Declarative Mappings

To obtain a semantic KG, a user must specify how the source data maps to a target vocabulary. Declarative mapping languages express this mapping as a set of rules. R2RML [9] is the W3C recommendation for mapping relational databases to RDF. RML [10] generalizes R2RML to heterogeneous sources such as CSV, JSON, and XML. Mapping engines such as Morph-KGC [3] execute these mappings and materialize the resulting KG at scale.

$$\text{Source Data}(D) + \text{RML Mapping} \xrightarrow{\text{Mapping Engine}} \text{Semantic KG } (G)$$

$$\text{Generic KG}(G_D) + \text{CONSTRUCT Query} \xrightarrow{\text{SPARQL Engine}} \text{Semantic KG } (G)$$

An alternative is to express the mapping as a SPARQL CONSTRUCT query. Tools like Tarql [8] apply a user-written CONSTRUCT query to each row of a CSV file. SPARQL Anything [4] generalizes this idea: it exposes data in many formats through a generic RDF view, called Facade-X, which the user then transforms into the target vocabulary with SPARQL CONSTRUCT queries.

Asprino et al. [4] argue that mapping languages require users to know the quirks of the source format, the target ontology, and a dedicated mapping syntax at the same time, whereas a generic façade combined with a general-purpose query language only requires knowledge of SPARQL. This approach is the closest predecessor to our work. Our approach follows the same architecture: a generic KG, transformed by a CONSTRUCT query. The difference is that, in SPARQL Anything, the query has to be written by the user, while we let an LLM agent generate it.

In all of these approaches, the mapping itself is written manually. The tools automate the execution of the conversion, but not the modeling decisions it encodes.

## 2.2 LLM-Based Knowledge Graph Construction

LLMs have recently been applied to many steps of KG construction, from ontology engineering to knowledge extraction and fusion [6]. They can read and write RDF and SPARQL with reasonable accuracy [12, 23].

For the conversion of structured data, two main directions have emerged: direct triple extraction and mapping generation.

### 2.2.1 Direct Triple Extraction

In the first direction, the LLM receives the source data, usually in chunks or row by row, and outputs RDF triples directly. Tuozzo et al. [30] does a small qualitative feasibility study to evaluate whether a GPT model can be used to convert a CSV file of KG quality metrics into RDF triples that conforms to the Data Quality Vocabulary.

Pellegrino and Tuozzo [24] extend this study to investigate the effectiveness of different prompting strategies with various LLMs. Due to input limitations, they restrict their experiments to 10 rows of the CSV file. They report that the models struggle with incomplete outputs, instability across runs, and hallucinations as the number of columns grows.

Hariri et al. [14] convert archaeological CSV data into RDF that conforms to CIDOC CRM. They split the data into chunks of 20 rows and compare the performance of various prompting strategies across three settings: no ontology, the full ontology, and a subset curated by a domain expert. The setting without an ontology performs worst, and the curated subset performs best.

Direct triple extraction has two inherent limitations. First, its cost grows with the size of the data, since the LLM is invoked for every chunk. Second, the LLM makes its modeling decisions anew for every chunk, so the same entity may receive different IRIs or types in different chunks. Moreover, no reusable artifact is produced that could be inspected, corrected, or executed again on new data.

It is also to be noted that in these approaches, the target ontology was given, so the task is ontology population, and the LLM is not deciding the semantic model from scratch. If an LLM agent has to make modelling decisions such as which tables become edges or which predicates to use, that is a harder task than the one these studies evaluate.

### 2.2.2 Mapping Generation

In the second direction, the LLM generates a declarative mapping, which a mapping engine then executes.

Hofer et al. [16] let LLMs generate RML mappings from a single JSON record of IMDB data and a hand-selected subset of the DBpedia ontology, to which they deliberately added confusable properties. Mappings with syntax errors are given back to the model together with the parser's error message for up to two repair rounds. Valid mappings are executed with an RML engine, and the resulting KG is compared with a manually created reference. Of five commercial models, only Claude 3 Opus and GPT-4 regularly produced mappings that yielded triples.

OntoGenix [31] goes one step further and also designs the ontology. Its agents first derive an ontology structure from a statistical summary of a CSV file, then write the ontology in Turtle, and finally generate an RML mapping that is executed with Morph-KGC. It experiments on 6 kaggle datasets but each dataset is only a single-csv file. Compared with ontologies built by humans for the same six datasets, the resulting ontologies are richer in annotations but structurally shallower, and they contain more modeling pitfalls.

Schmidt et al. [28] generate YARRRML and RML mappings for three manufacturing

data sources at Bosch, exported from relational databases as CSV files, against small existing ontologies. They vary how much data the model sees (column headers only, or the first 20 or 200 rows), how the ontology is reduced before prompting, and whether the prompt contains a mapping template or retrieved examples. Compared with expert-written mappings, the best configurations reach a relaxed triple F1 of about 0.7, while exact matching drops sharply because the generated IRIs differ from the reference.

Trajanoska et al. [29] map relational data to schema.org with a multi-agent system that processes one table at a time. A mapping agent assigns schema.org terms to columns, using candidate terms retrieved from an index over the vocabulary. A relation agent identifies primary and foreign keys and creates links between entities, and a validator agent reviews both steps. The output is an assignment of terms rather than an executable mapping, and its accuracy was judged manually on databases from five domains of the Spider benchmark.

With the exception of OntoGenix, these approaches assume that a target ontology is given, and the LLM only selects terms from it. Moreover, they are evaluated on small datasets: a single JSON record, single CSV files, or a fixed set of test cases.

## 2.3 Competency-Question-Driven Modeling

Several approaches use competency questions to guide LLM-based ontology engineering. CQs were introduced by Grüninger and Fox [13] as natural-language questions that an ontology must be able to answer.

The central idea is that the design and evaluation of an ontology are the same activity: CQs define the requirements up front and then serve as the test that the axiomatised ontology must pass. Later methodologies kept CQs as their requirements mechanism but changed how they are checked. Instead of proving logical entailment, as Grüninger and Fox did, they test CQs as queries against an implemented ontology, typically in SPARQL. This later, query-based sense is the one used in this thesis.

With LLMs, CQs have also become an input to ontology generation, translating natural-language questions into formal ontologies. Ontogenia [19] lets an LLM output an OWL ontology using 14 CQs about African wildlifeas input. There is no data, the CQs are the only description of the domain. The model first interprets each CQ, then splits it into subject, predicate, and object to identify classes and properties, extends the ontology with restrictions, and finally justifies its decisions and tests the ontology on example instances.

Saeedizade and Blomqvist [27] generate OWL ontologies from ontology stories and their CQs. They point out that such input describes only the desired outcome, not how to model it. They compare zero-shot prompting, chain-of-thought variants, and two decomposition strategies across a range of models, and find that only GPT-3.5 and GPT-4 produce usable OWL. Their CQbyCQ technique processes one CQ at a time and merges each result into the growing ontology.

Evaluation is CQ-based: for each CQ, they check whether a SPARQL query answering it can be written against the generated ontology. This is particularly relevant for the purpose of evaluation in our experiments as well, as seen in section todo. Ignoring minor errors, GPT-4 with CQbyCQ comes close to the submissions of their student groups. It does particularly well on object properties and reified relations, but less well on restrictions. A follow-up study [20] develops both techniques further and reports that o1-preview with Ontogenia outperforms novice ontology engineers.

Taken together, these works suggest that CQs are a promising tool for our setting, in both of the roles that Grüninger and Fox [13] intended. As an input, CQs have proven to be a workable basis for LLM-based modeling and even as an evaluation criterion, CQs give a direct and interpretable measure of whether a model serves its purpose, whether checked by writing a query for each question [27], by scoring extracted triples against domain questions [14], or by executing paired queries over the source database and the generated graph [34].

Since CQs describe what a graph should be able to answer rather than how it should be modeled, they apply to the conversion of existing data just as they apply to designing an ontology from scratch. This makes them a natural fit for the modeling decisions that the source schema leaves open. So far, however, CQs have been given to LLMs as input only when designing ontologies, and the studies always included them, so their effect on the modeling has not been isolated. In our thesis, we aim to extend this line of work to the conversion of structured data and test directly whether supplying CQs changes the LLM's modeling decisions.

## 2.4 Evaluation in Related Work

Related works evaluate LLM-based KG construction at several levels. At the syntactic level, the output is parsed, for example with rdflib. Martin et al. [21] sort outputs into valid, recoverable, invalid, and empty.

Hofer et al. [16] additionally give the model up to two rounds to repair invalid outputs. At the level of the graph, the output is compared with a manually created reference. This comparison uses precision and recall of triples [14, 21] or the F1 score of exact graph matches [16].

Pellegrino and Tuozzo [24] count the correctly identified entities, dimensions, and metrics, and repeat every experiment five times to measure stability. BLINKG evaluates individual mapping decisions instead of the resulting graph [7].

When the LLM also designs the ontology, its quality is assessed with structural metrics such as OQuaRE and with the pitfall scanner OOPS! [31, 17]. Finally, Kommineni et al. [17] and Hariri et al. [14] test whether the generated KG can answer a set of competency questions.

Comparisons with a reference KG presuppose a fixed target vocabulary. Since we do not give a target ontology, a semantic KG produced by our approach may use different, but equally valid, classes and predicates than any reference. We therefore evaluate against the criteria defined in Section 1.2. Where possible, we use objective measures for fidelity, such as coverage and soundness. For the quality of the semantic modeling, we use a qualitative assessment, and for usefulness, we test whether competency questions can be answered.

# 3 Background

This chapter introduces the relevant concepts that our approach uses or builds upon.

## 3.1 SPARQL CONSTRUCT Queries

The central artifact of this thesis is a set of SPARQL CONSTRUCT queries: our approach generates these queries, and executing them on the input generic KG yields the semantic KG. The quality of the output therefore depends entirely on the quality of the queries. Before describing how our approach generates such queries, we explain how a CONSTRUCT query is structured.

SPARQL [15] supports four query forms, one of which is the CONSTRUCT form. It returns an RDF graph constructed by substituting variables in a set of triple templates.

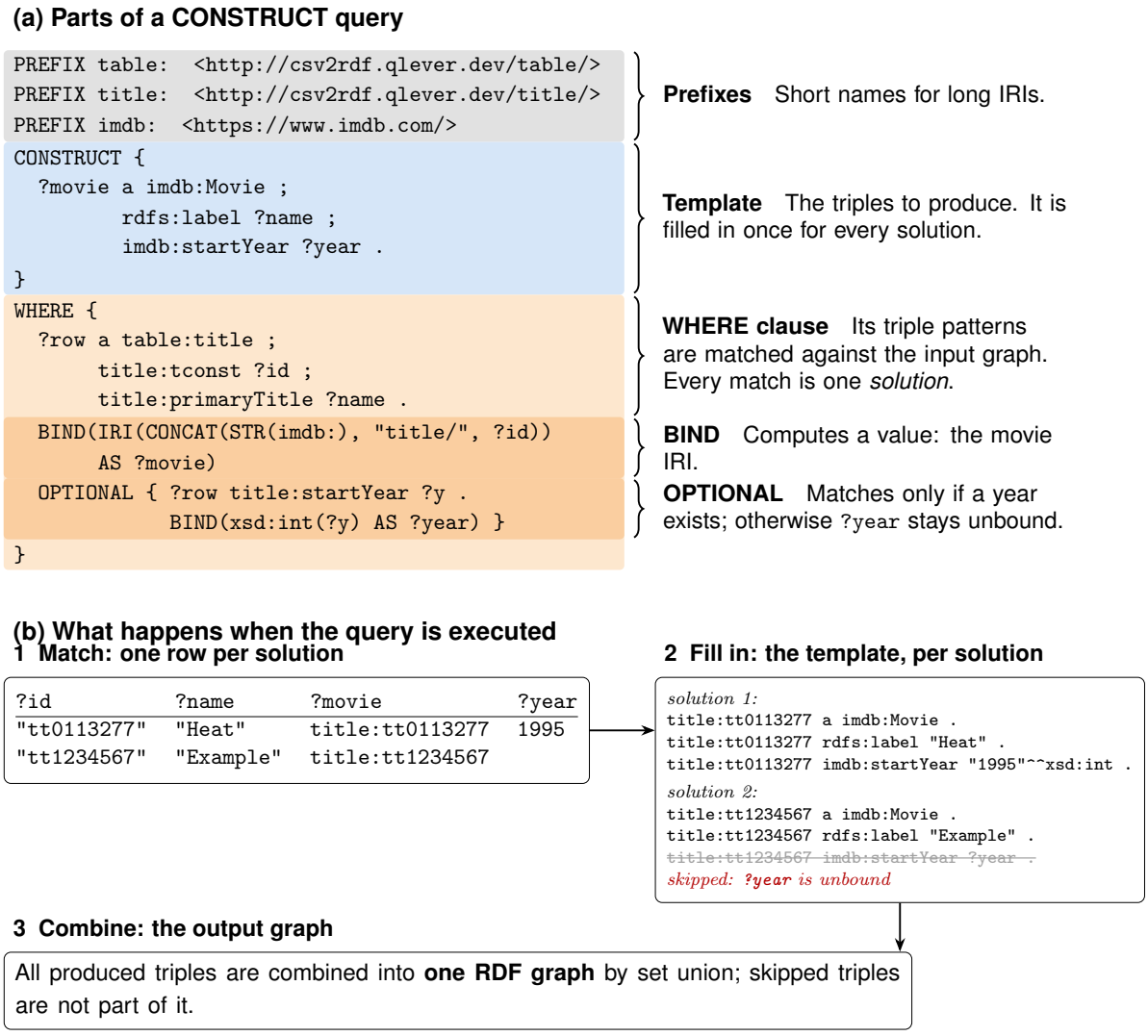



**Figure 4:** A CONSTRUCT query over a sample RDF KG

As shown in Figure 4, a CONSTRUCT query consists of two parts:

- a *template* which describes the triples that the query produces

- a WHERE clause which describes which data the query reads from the input graph. Its triple patterns are matched against the input graph.

For every solution of the WHERE clause, the variables in the template are replaced by their values, and the resulting triples are added to the output graph. The output graph is the set union of these triples.

So, in the example given in Figure 4, the template states that a movie has the type `imdb:Movie`, a label, and a start year. In the WHERE clause, the first pattern selects every node of class `table:title`, that is, every row of the table `title` in the generic KG. The other two patterns read the identifier of each such row into `?id` and its title name into `?name`. All three patterns must match for a row to be selected.

*BIND* computes a new value from values that were already matched, and assigns it to a new variable. In the figure, BIND appends the identifier of the movie to the IMDb namespace and converts the resulting string into an IRI with the function `IRI`. This is how a query mints IRIs for the entities it creates.

*OPTIONAL* marks a part of the WHERE clause that may fail to match without discarding the entire row. In the figure, the start year is optional. If a row has a start year, `?y` is bound to it, and the inner BIND converts it into an integer `?year`. If a row has no start year, the row is still selected, but `?y` and `?year` remain *unbound*, that is, they have no value. In this case, the template triple that uses `?year` is skipped for this row, and the output graph contains no triple about the start year of this movie.

**Execution:** Figure 4b shows how a SPARQL engine executes the query on two rows of the input graph. Execution proceeds in three steps [15].

1. *Matching.* The engine finds all ways in which the WHERE clause matches the input graph. Each match is called a *solution* and assigns a value to every variable that the match binds. In the figure, the two rows yield two solutions. The second solution leaves `?year` unbound, because its row has no start year.

2. *Instantiation.* For each solution, the engine replaces the variables in the template by their values. Each solution thus yields a copy of the template with concrete values. If a template triple contains a variable that is unbound in this solution,

that triple is left out, while the other triples of the same solution are kept. In the figure, this happens to the start-year triple of the second solution. A triple is also left out if it is not valid RDF, for example if a literal ends up in the subject or predicate position. Template triples without any variables are copied into the output unchanged.

3. *Combination.* The engine combines the triples of all solutions into one RDF graph by set union.

**Caveats:** While the execution of a CONSTRUCT query is straightforward, it has some caveats that are important for our approach.

- *Required patterns discard whole rows.* Every triple pattern of the WHERE clause that is not inside OPTIONAL must match [15]. If a row has no value for such a pattern, the row yields no solution, and none of its triples appear in the output. In the figure, a title without a primary title would therefore disappear entirely. The same happens if a predicate in the WHERE clause is misspelled: the pattern matches nothing, and every row is discarded. Values that may be missing must therefore be read in OPTIONAL.

- *IRIs must be minted consistently.* In RDF, two different IRIs are two different nodes. A query therefore refers to the same entity from several places only if it computes the same IRI each time. If two parts of a query mint the IRI of a person with different expressions, the output contains two separate nodes, and nothing in the graph indicates that they describe the same person.

- *Blank nodes are fresh per solution.* A blank node in the template denotes a new node for every solution [15]. Triples from different solutions therefore never share a blank node. Blank nodes are thus unsuitable for entities that are referenced from several solutions or several queries.

A CONSTRUCT query is executed by a SPARQL engine. In this thesis, we use QLever [5], which loads the generic KG $G_D$ and materializes the output graph $Q(G_D)$. The size of the query depends only on the schema of $D$, not on the number of rows. The query is therefore a compact representation of the entire conversion, which can be read, corrected, and executed again on new data.

## 3.2 LLM Agents and Tool Calling

An LLM agent is an LLM that solves a task by interleaving reasoning with actions in an environment, and by observing the results of these actions [35]. In current LLM APIs, actions are implemented as *tool calls*. The agent is given a set of tools, each described by a name, a description, and a schema of its parameters. In each step, the LLM either produces a final answer or emits a call to one of the tools with required arguments. The runtime executes the call and returns its result to the LLM as a new message, and the LLM continues with the next step. This loop ends when the LLM produces its final answer or when a maximum number of steps is reached.

The capabilities of an agent are thus determined by its tools. A *general-purpose AI coding agent* is equipped with generic tools: reading and writing files, searching a code base, and running shell commands. With these tools, it can solve a wide range of tasks, for example by writing and executing scripts. OpenCode [1] is an open-source coding agent of this kind, which runs in the terminal and supports models from many providers. One can also configure their own LLM instances to work with the agent.

## 3.3 GRASP

GRASP [33] is an LLM-based agent originally designed for generating SPARQL queries over RDF knowledge graphs. Its central idea is that the agent is not given a description of the KG in advance, but explores it itself. Through a set of tools, the agent searches indices of the entities, properties, and literals of the KG, lists the triples of individual entities, and executes SPARQL queries. In this way, it learns how the KG is structured and what data it contains, before it writes its final query. In this thesis, we aim to extend GRASP with a new task, in which the output is a set of CONSTRUCT queries instead of a SELECT query. In the following, we describe the features of GRASP that this task particularly uses.

**Tools:** The agent explores the input KG with a set of tools. The functions `search_entity`, `search_property`, and `search_literal` search for specific data in the KG. For this, GRASP builds three search indices per KG: a fuzzy index for entities and an embedding index each for properties, and for literals. The function `search_property_of_entity` searches the properties used by a given entity, and `search_object_of_property` searches the common objects of a given property. The function `execute` runs an arbitrary SPARQL query and returns a preview of its results. If a shape index is available, the functions `search_shape` and `get_shape` additionally return the shapes of classes.

**Shapes:** A shape index describes the class structure of the input KG. Typically, for each class, the shape lists the properties that its instances use, together with the type of their values: a datatype for literals, or, for properties that point to other entities, the class of these entities. In addition, the shape lists the incoming properties of the class in a separate list (`incoming`): properties of other classes whose values are instances of this class, together with the class they come from.

```
{
  "iri": "<table/person>",
  "profile": {
    "total_entities": 4,
    "properties": [
      {
        "iri": "<person/id>",
        "triple_count": 5,
        "entity_count": 4,
        "targets": [{ "type": "literal", "datatype": "xsd:int", "triple_count": 4 }]
      },
      {
        "iri": "<person/name>",
        "triple_count": 4,
        "entity_count": 4,
        "targets": [{ "type": "literal", "datatype": "xsd:string","triple_count": 4 }]
      },
    "incoming": [
      {
        "iri": "<ref/movie/director_id>",
        "triple_count": 745,
        "entity_count": 3,
        "targets": [{ "type": "class", "iri": "<table/movie>","triple_count": 745 }]
      }]
  },
}
```

**Figure 5:** GRASP sample shape format for one class

**Notes:** Before the agent is used on a KG, GRASP also has the feature to let it explore the KG freely and record its observations as notes. These notes may then be used in other tasks so that that it starts with knowledge that would otherwise have to be gathered again with exploration. Figure 6 shows sample notes recorded for a database related to superheroes. They describe both schema and data characteristics.

1. Main class structure: `person` has `id`, `person_name`, `full_name` (optional), `height_cm`, `weight_kg`, and foreign key references to `colour(eye/hair/skin)` , `gender`, `race`, `publisher`, and `alignment` tables.
2. Naming patterns: Tables use `table/{name}`. Rows use `row/{table}/{id}` or `_rowN` for junction tables. Properties: `ref/{table}/{fk}_id` for foreign keys, `{table}/{field}` for direct properties.

**Figure 6:** GRASP sample structural notes

# 4 Approach

In Section 1.2, we defined our task: given the generic KG $G_D$ of a structured dataset $D$, find a set of CONSTRUCT queries $Q$ such that $Q(G_D)$ is a semantic KG of $D$. This chapter describes how we propose to implement this task.

Our approach follows the idea of GRASP (Section 3.3): instead of giving an LLM a description of the data, we let an LLM agent explore the data itself, and write its result as SPARQL queries. The difference to the original use of GRASP is the kind of query. The agent does not write a SELECT query that answers a question, but CONSTRUCT queries that remodel an entire KG.

Figure 7 gives an overview. The approach consists of four steps, each described in its own section:

1. **Convert** $D$ into a generic KG $G_D$ with a direct mapping (Section 4.1).

2. **Prepare** $G_D$ for exploration by building the notes, search indices, and shapes of GRASP (Section 4.2).

3. **Generate** the CONSTRUCT queries with a generation agent, and check them with an independent review agent (Section 4.3).

4. **Materialize** the semantic KG by executing the queries on $G_D$.

Finally, Section 4.4 describes the baseline against which we compare our approach.

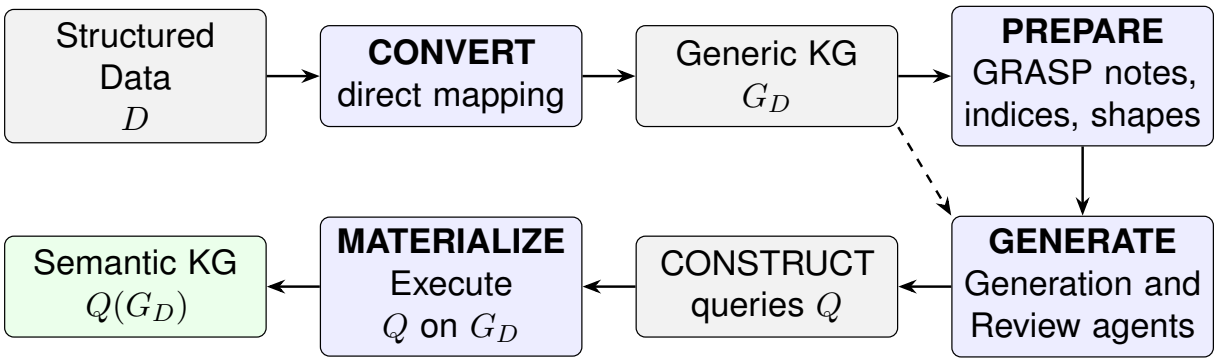



**Figure 7:** Overview of generic-to-semantic KG pipeline

## 4.1 Structured Data to Generic KG

Our task, as defined in Section 1.2, starts from an RDF KG that is to be modeled semantically, regardless of the format in which the data was originally stored. The first step therefore, is to convert the source data $D$ into a generic KG $G_D$. Converting every source into a generic KG first keeps the rest of the pipeline independent of the source format: the agent explores the data with SPARQL, and writes its result as SPARQL CONSTRUCT queries over the same graph.

**Mapping:** In this thesis, we only consider relational data for which we refer the W3C Direct Mapping [2] to generate the generic KG.

- The row becomes a node whose IRI consists of a base IRI, the table name, and the value of the primary key. If the primary key is missing, the row is identified by its position, as `_row`$N$. A composite primary key is written as `column=value` pairs joined with `;`.

- The row receives an `rdf:type` triple with a class named after its table.

- Every non-null cell yields a triple with a predicate named after the table and the column.

- Where the source declares column types, we follow R2RML [9]: integer types map to `xsd:integer`, `NUMERIC` and `DECIMAL` to `xsd:decimal`, floating-point types to `xsd:double`, and boolean, date, and timestamp types to `xsd:boolean`, `xsd:date`, and `xsd:dateTime`. A value receives a datatype only if it also matches that datatype's lexical form; otherwise, and for all other cases, it is written as a plain string.

- For a declared foreign key, the row receives an edge to the referenced row and optionally may also emit a triple with the foreign key value as a literal. For composite primary keys, the foreign key value is emitted accordingly to that single node.

If a schema is known, the converter additionally emits, once per table, a class declaration and the `rdfs:domain` and `rdfs:range` of each predicate.

Table 1 summarizes the mapping rules and gives examples.

| Source | Generic KG | Example |
|---|---|---|
| Row | `<.../row/{table}/{key}>` | `<row/title/tt0113277>` |
| Table | `rdf:type <.../table/{table}>` | `<table/title>` |
| Cell | `<.../{table}/{column}>` | `<title/primaryTitle>` `"Heat"` |
| Foreign key | `<.../ref/{table}/{column}>` | `<ref/principals/nconst>` |
| Typed column | XSD datatype, if the declared type maps to one and the value is valid | `"1995"^^xsd:integer` |

**Table 1:** Rules of the direct mapping

**Conversion Practices:** Real-world databases deviate from the idealized picture of the Direct Mapping in several ways. The converter therefore applies the following practices to all datasets:

- **Sanitizing special characters:** Table names, column names, and key values become parts of IRIs, but may contain characters that IRIs do not allow. Every character outside the unreserved set of RFC 3986 (ASCII letters, digits, and `-` `.` `_` `~`) is percent-encoded as its UTF-8 bytes.

  `image/jpeg`, a primary key $\xrightarrow{\text{percent-encode}}$ row IRI `<row/image_type/image%2Fjpeg>`

- **Null handling:** Values are trimmed of leading and trailing whitespace, and a cell yields no triple if it is then NULL or empty. By default, the converter treats only real NULL values and empty strings as missing. In addition, it accepts an optional list of null markers, that is, strings that stand for a missing value in a particular dataset. Text files such as CSV or TSV files, for example, cannot represent NULL, and therefore often write missing values as `\N`. Values that only mean “unknown” within the domain, such as `-` or `N/A`, are not interpreted and remain ordinary values, since deciding their meaning is a modeling decision left to the agent unless the dataset owner explicitly declares them as null markers.

- **Multi-valued cells:** Delimited files often contain cells with multiple values either separated by a delimiter or in an array, for instance, a list of genres `Action,Crime,Drama`. Besides the field delimiter of a file, the converters therefore accept an optional sub-delimiter, together with the columns it applies to, or all columns if none are given. A cell in such a column is split into one literal per value.

- **Sampling:** An agent does not need to see every row of a table to understand its structure, and a large KG slows down exploration. The converters can therefore optionally produce a sample of the data instead of converting it in full.

  Sampling is controlled by three parameters: a size threshold, a sample size, and a limit for foreign-key repair. Tables below the threshold (by default 5,000 rows) are always converted completely. From larger tables, a fixed number of rows (by default 2,000) is drawn uniformly at random using reservoir sampling [32].

  For datasets where schema is available, sampling the tables independently, breaks references: a sampled row may refer to a row in another table that was not part of that table's sample. To fix this, we apply *foreign-key repair*. While sampling, the converter records every value that a foreign key refers to. It then adds the rows that hold these values to their tables. Each row is looked up by the column that the foreign key names(which may not always be the primary key). References from completely converted tables are also recorded, because they can point into sampled tables as well.

  To keep the sample small, repair is limited in two ways. First, it goes only one level deep, so the references of the added rows are not followed. Second, it adds at most a fixed number of distinct values per referenced column (by default 1200). References beyond these limits remain dangling in the sampled KG.

  Reservoir sampling reads the table in a single pass and needs memory proportional to the sample size, not to the size of the table. If a random seed is given, the sample is reproducible.Moreover, sampling does not limit the result. A CONSTRUCT query set generated on a sample can be executed on the generic KG of the complete data.

Beyond these general practices, some datasets required specific handling. We describe these cases together with the datasets in Section 5.1.

## 4.2 Preparing the Generic KG

A generic KG of a database with hundreds of tables is too large to understand by browsing its triples. Before the agent starts, we therefore let GRASP build the components described in Section 3.3. We load $G_D$ into a QLever instance [5], which serves as the SPARQL endpoint for all further steps, and run the setup of GRASP once per dataset, as per the following:

1. **Info.** The setup task determines the prefixes of $G_D$ and writes a short description of the input KG.

2. **Structural notes.** Over three rounds, the agent repeatedly picks a class or property in a part of $G_D$ that is not yet well covered, explores its connections, and records notes on the structure and content of $G_D$, such as those in Figure 6.

3. **Search indices.** The setup task writes the SPARQL queries that select the entities, properties, and literals of $G_D$. GRASP executes them and builds the three search indices.

4. **Shapes.** Given the structural notes and search indices, the setup task determines how $G_D$ assigns classes to instances and declares its schema. GRASP then discovers the classes, profiles their instances, and builds the shape index, with shapes such as the one in Figure 5. Since every row of the generic KG is typed with the class of its table, the setup reliably finds the class of every row, and each shape summarizes one table.

## 4.3 Generating CONSTRUCT Queries

The core of our approach is the generation of the CONSTRUCT queries. The manual process it replaces consists of understanding the data, designing a model, and writing and checking the mapping. We divide this work between two agents, both implemented as tasks of GRASP and equipped with its exploration tools:

- a *generation agent* (`construct-gen`), which explores $G_D$, designs the model, and writes the queries,

- and an independent *review agent* (`construct-review`), which checks the result against the data.

Figure 8 shows how the two agents interact. The generation agent writes and tests queries until it is satisfied, and then submits them as a set. Every accepted submission is checked by the review agent, which returns a verdict together with feedback. Unless the verdict states that the queries are done, the generation agent revises them and submits again, up to a configured maximum number of review rounds.

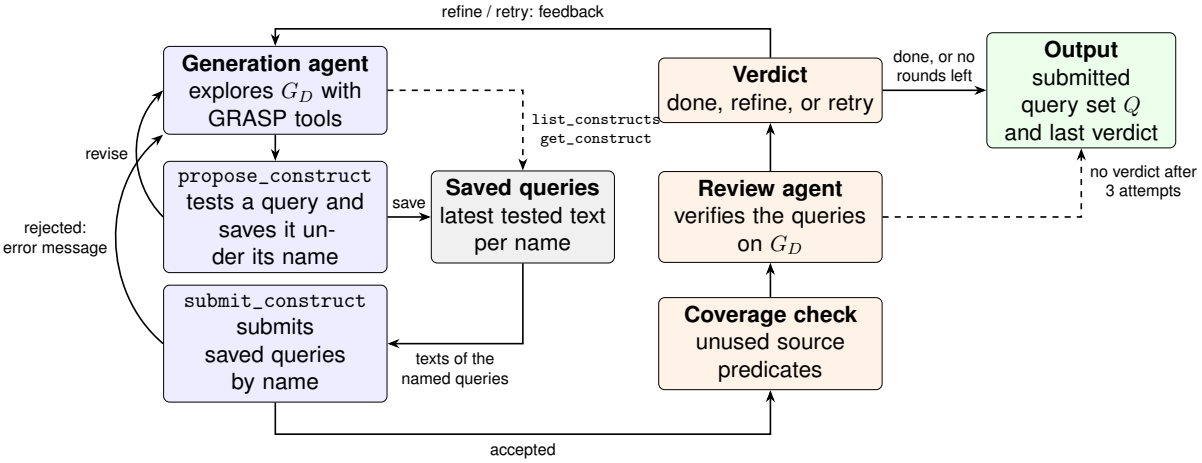



**Figure 8:** Generation and review of CONSTRUCT queries.

In the following, we first describe the generation agent and how it manages its queries, and then the review agent and the loop between the two.

**Generation agent:** The prompt of the generation agent describes the task: analyze the input generic KG, $G_D$ and produce CONSTRUCT queries that model it as a well-formed, semantically correct KG. Since $G_D$ may already model some of its data well, the agent should keep or refine such parts, and remodel the rest. The prompt asks the agent to proceed in the following steps, which mirror the manual process:

1. **Explore** $G_D$, starting from the notes and shapes, and run targeted queries, if necessary, to fill gaps in its understanding of the source data.

2. **Design** the mapping. Decide which entities to model, which classes to assign, and which fields are relationships between entities. Mint IRIs from stable identifiers, in a namespace that reflects the domain. Determine how missing values look in the data, and how to handle fields where units, currencies, or scales may be attached. Finally, decide how many queries are needed to cover all the data without loss.

3. **Test** each query under a short, descriptive name, and check that it runs and produces well-formed output.

4. Optionally, **check** the design against the competency questions, if given, by sketching a SPARQL query for each question over the graph the queries would produce. A question that cannot be answered, or only with an overly complex query, points to a gap in the model.

5. **Submit** the queries once they are complete.

The competency questions are presented as a minimum, not as the scope of the model: the agent must not narrow the model to the questions at the expense of coverage.

**Rules:** In initial experiments, we observed several recurring errors in the generated queries. We therefore added the following rules to the prompt to steer the agent away from them:

- Every entity should receive an `rdfs:label`, but the label must not be a required pattern, since a missing label would otherwise drop the entity.

- Values that are actually missing should be read in OPTIONAL, so that they do not suppress the other triples of the entity. OPTIONAL however, should not be used as a blanket pattern for values that are always present.

- An entity referenced by several queries must be minted with the same IRI pattern in each, and the same predicates and namespace be used for the same concept, so that the queries combine into one coherent graph.

- A unit, currency, or scale encoded in a column name, as in `height_cm`, should ideally not be lost while conversion.

**Generation Agent Tools:** In an early version of our approach, the agent submitted its queries by writing out their full text. This caused two problems. First, the agent had to retype queries it had already tested, and could introduce new errors in doing so. Second, when resubmitting after review feedback, it sometimes left out queries it had not changed, and their data silently disappeared from the output.

We therefore separate testing a query from submitting it. The generation agent works with two collections: the *saved queries*, which hold the latest tested text of every query under its name, and the *submitted set*, which is the final set of queries that are submitted

for review. Four tools operate on these collections:

- `propose_construct` takes a name and a query, saves the query under the name, and tests it on $G_D$. It returns up to 20 of the produced triples, or the error message.

- `list_constructs` lists all saved queries, with whether each is submitted and whether its latest test succeeded.

- `get_construct` takes a name and returns the exact saved text of the query.

- `submit_construct` takes the names of all queries that form the set, and replaces the submitted set by these saved queries. It reports which queries were added, updated, and dropped.

A query goes through the following life cycle:

- **Propose.** The agent proposes a new CONSTRUCT query under a name and saves it. The agent can propose a query with the same name to update it, and the saved text is always the latest version.

- **Test.** The agent tests the proposed query with `propose_construct`. A test succeeds if the query runs and produces at least one triple. If the query fails or produces no triples, the agent is asked to revise the query. Since the saved text will later be submitted, a `LIMIT` or `OFFSET` that the agent adds itself would truncate the final output. It is therefore removed from the saved text, and the agent is informed.

- **Inspect.** The agent can check which queries it has saved and submitted, and read the exact text of a saved query, for example to reuse its IRI patterns in another query.

- **Submit.** The agent submits by calling `submit_construct` with the names of all queries that form the set. From the saved queries, it creates the submitted set as per these names. If a name is empty, duplicated, has no saved query, or belongs to a query whose latest test failed, the call is rejected, and the submitted set remains unchanged. An accepted submission starts the review.

- **Revise.** After review feedback, the agent tests revised versions of its queries under

their existing names, and submits the set again. Queries that it did not revise keep their saved text, and are submitted unchanged.

- **Delete.** A previously submitted query whose name is left out of a new submission is dropped from the set, for example when its data is now produced by another query. The agent is asked to state the reason in the argument `removed`: why the query is no longer needed, and where its data is covered now. The reasons are collected across all review rounds.

This life cycle resolves both problems of the earlier version. The submitted query is always the one that passed the test, since the agent never retypes a query, and no query disappears unnoticed: every submission reports the dropped queries, and dropping a query requires a deliberate decision, ideally with a stated reason.

**Review agent:** Iterative refinement, in which an LLM revises its output based on feedback, is an established technique for improving LLM output. However, when an LLM only reconsiders its own output, such self-correction often fails to improve the result. The submitted queries are therefore checked by an independent review agent. It has the same exploration tools and access to $G_D$ as the generation agent, but not its trace, so it does not inherit the generation agent's assumptions. The reviewer is instructed not to trust the queries as written, but to verify every aspect by querying $G_D$, and to report only issues that it has verified.

Every accepted submission starts a review round, which proceeds in three stages. First, the coverage of the source data is computed mechanically. Then, the review agent checks the submitted queries against a checklist, with the computed coverage as additional input. Finally, the reviewer returns a verdict, which decides how the run continues. We describe the three stages in this order.

**Computing source coverage.** Of all aspects that the reviewer has to check, source coverage is the most important. As per our initial experiments, it was clear that as the schema size increases, it becomes more likely that the agent misses a lot of the columns from the source data, in its proposed queries. Before the reviewer starts, we therefore compute coverage mechanically by comparing the predicates of $G_D$ with the predicates that the submitted queries mention.

1. **Listing the source predicates.** A SPARQL query lists every predicate of $G_D$, together with its number of triples. Since every column of a table becomes a

predicate in the generic KG, this list corresponds to the columns of $D$.

2. **Collecting the referenced IRIs.** For every submitted query, all IRIs that the query mentions are collected. These are full IRIs and prefixed names, which are expanded using the prefix declarations of the query. The keyword `a`, the SPARQL shorthand for `rdf:type`, counts as a reference to `rdf:type`.

3. **Comparing.** A source predicate counts as covered if at least one query mentions it.

The result is added to the reviewer's prompt: the number and percentage of covered predicates, and the list of uncovered predicates, ordered by their number of triples, so that the predicates that carry the most data come first. The check is an upper bound on the actual coverage, since a predicate that a query mentions may still lose values, for example through a filter or a missing OPTIONAL. But it is intended to guide the review agent's attention so that it can focus on the most important issues first. If the source predicates cannot be listed, for example because the endpoint fails, the reviewer is told to check coverage itself.

**Checklist.** The review agent receives the submitted queries, the computed coverage, and a checklist of aspects to verify:

- **Source coverage.** All data of $G_D$ is accounted for by the queries, and nothing significant is missing. For every uncovered predicate in the computed coverage, the reviewer reports whether it matters and what should be added to cover it, unless its data is represented in some other way.

- **Value types.** For each predicate in the output, the type of its values is appropriate: a literal where a value is expected, and an IRI where a relationship to another entity is intended.

- **Relationship modeling.** The reviewer should first query $G_D$ to independently identify which relationships exist between entities in the data, and then checks that the queries model them. Every field that refers to another entity must become an IRI link to that entity, not remain a raw literal, regardless of whether the entity is minted in the same query or in a different one.

- **Consistency.** The same real-world entity is minted with the same IRI pattern wherever

it is referenced, within a query and across queries.

- **Namespace consistency.** All custom predicates and entity IRIs use one namespace, which reflects the domain of the data. The reviewer judges this from the full IRIs, not from the prefix names used in the queries.

Depending on the submission, the checklist contains further items. If competency questions are given, two items are added:

- **Competency question coverage.** For each question, the reviewer writes the SPARQL query that would answer it over the graph that the queries produce. It cannot execute this query, since that graph does not exist yet, but writing it serves as a usability test of the model: a question that cannot be answered, or only with an overly complex query, points to a gap in the model, and the reviewer names the part of the model responsible.

- **Competency question scope.** It is specified to the reviewer that the questions are a minimum, not the full scope of the model. The reviewer checks that the model was not narrowed to the questions at the expense of source coverage.

If several queries are submitted, one more item is added:

- **Cross-query consistency.** The queries form one coherent graph: the same entity is named and modeled consistently across queries, the same predicates and namespace are used for the same relationship, and every entity type referenced by more than one query is minted with the same IRI pattern.

**Verdict and review loop.** The reviewer ends its round with one of three verdicts, together with feedback that describes the issues it found: *done*, if the queries are correct and complete; *refine*, if they need targeted fixes; or *retry*, if the overall approach must be reworked. For *refine* and *retry*, the feedback is passed to the generation agent as a new message, and the agent revises and resubmits its queries as described above, which starts the next round. The loop ends when the verdict is *done*, or when the configured maximum number of review rounds is reached. In the latter case, the output is marked as submitted with unresolved issues, so that such runs can be distinguished in the evaluation.

The reviewer is itself an LLM agent and can fail: its run may end with an error, or without a verdict. Such a failure says nothing about the queries, so the reviewer is run again on the same queries, up to three times. These attempts do not count as review rounds. Only if all attempts fail are the queries output without a review, marked as unreviewed.

**Output.** A run produces the submitted set of named CONSTRUCT queries, the reasons given for dropped queries, and the last verdict, with its feedback, the number of review rounds, and whether the review budget was exhausted. We also store the complete traces of both agents, which we use to analyze the behavior of the agents in Chapter 5.

## 4.4 Baseline

We compare our approach with a baseline that uses a general-purpose coding agent for the same task. We use OpenCode [1], an open-source agent that runs in the terminal and works on the files of a project directory (Section 3.2). Its tools are generic: it can read and write files, search them, and run shell commands.

**Task and input:** The baseline solves the same task as our generation agent: it writes a set of CONSTRUCT queries that transform the generic KG $G_D$ into a semantic KG. Unlike our approach, it does not access $G_D$ through a SPARQL endpoint, but receives it as an N-Triples file, the same sample that our approach explores via a SPARQL endpoint. In the corresponding configuration, it additionally receives the competency questions. It writes its queries as files into a given output folder, and refrains from accessing any other files.

Since it's a general-purpose coding agent, the baseline has to explore the data with its generic tools, for example by searching the N-Triples file for the predicates of a table. For the same reason, it cannot directly execute the CONSTRUCT queries it writes. Whether and how it tests them is left to the agent so as to not constrain its behavior.

**Prompt:** To make the comparison fair, the prompt of the baseline mirrors that of our generation agent wherever possible. It describes the same task, asks the agent to proceed in the same steps, and contains the same rules (Section 4.3). Since the baseline has no independent reviewer, its prompt additionally contains the reviewer's checklist as a self-review: before finishing, the agent is asked to verify its queries against the input file, as per the given checklist.

**Differences to our approach:** The baseline thus receives the same data, task, and instructions, and produces the same kind of output. It differs in three ways:

- **Exploration.** Unlike our approach that can leverage the exploration tools, notes, search indices, shapes provided by GRASP, the baseline explores the data only as a file, with generic tools.

- **Testing.** The baseline is free to test its queries for example using rdflib or Oxigraph to run them on the N-Triples file, but it does not have the same testing as our approach, where the agent can test each query on the SPARQL endpoint and receive a sample of the produced triples or an error message and revise the query until it runs successfully.

- **Review.** It is upto the baseline agent to decide how to generate and test the queries. Its queries are not checked by an independent reviewer with a mechanical coverage check, but only by its own self-review.

Since both approaches produce CONSTRUCT queries over the same generic KG, we materialize and evaluate their outputs in the same way. The CONSTRUCT queries are executed on the QLever endpoint of $G_D$, and the resulting triples are written to an N-Triples file.

# 5 Evaluation

In this chapter, we evaluate our proposed approach against the baseline. We first describe the datasets, then the experimental and evaluationsetup, and finally report the results.

## 5.1 Datasets

We evaluate on eight relational datasets. Five come from the BIRD benchmark [18], and three are public datasets of varying schema sizes: IMDb, Mondial, and MusicBrainz. They were chosen to cover a range of domains and of schema sizes, from three tables to 239. Table 2 gives an overview.

| Dataset | Format | Tables | Columns | Rows | FKs | CQs |
|---|---|---|---|---|---|---|
| IMDb | TSV | 7 | 39 | 207.4 M | 7‡ | – |
| Mondial | SQLite† | 47 | 192 | 56,702 | 74 | – |
| MusicBrainz | TSV + DDL | 239 | 1,588 | 25.1 M | 473 | – |
| superhero | SQLite | 10 | 31 | 10,614 | 11 | 16 |
| california_schools | SQLite | 3 | 89 | 29,941 | 2 | 9 |
| formula_1 | SQLite | 13 | 94 | 493,257 | 19 | 20 |
| card_games | SQLite | 6 | 115 | 803,445 | 4 | 16 |
| european_football_2 | SQLite | 7 | 199 | 222,796 | 31 | 16 |

**Table 2:** Overview of the datasets. *FKs* counts declared foreign keys. *CQs* gives the number of competency questions given to the agents, out of all questions of BIRD mini-dev for that database. † converted from PostgreSQL. ‡ declared by us using the schema documentation.

### 5.1.1 BIRD Databases

BIRD [18] is a text-to-SQL benchmark of 95 SQLite databases from 37 professional domains. Each database comes with natural-language questions, each paired with a gold SQL query, a hint (called *evidence*) that explains the domain knowledge the question needs, and a difficulty rating of *simple*, *moderate*, or *challenging*. We use its curated subset mini-dev, which contains 11 databases, and from these we select five: superhero, card_games, formula_1, european_football_2, and california_schools.

- **superhero** describes 750 superheroes with their publisher, race, gender, alignment, eye, hair, and skin colors, and two many-to-many tables that link heroes to superpowers and to attribute values. It is the smallest database, and the only one in which two tables (`hero_power`, `hero_attribute`) have no primary key, so their rows are identified by position (`_row`$N$).

- **card_games** describes the cards of the trading card game *Magic: The Gathering*. The table `cards` has 74 columns, and several of them hold lists in a single string, such as the sets in which a card was printed. All four foreign keys reference a non-key column (`cards.uuid`, `sets.code`).

- **formula_1** describes Formula 1 seasons, races, circuits, drivers, and constructors, with results, qualifying, lap times, pit stops, and championship standings. It has the most tables among the BIRD databases (13), the most foreign keys (19), and two tables with a composite primary key.

- **european_football_2** describes European football leagues, teams, players, and matches, with yearly attribute snapshots of players and teams. Its table `Match` has 115 columns: 22 lineup slots (`home_player_1` to `away_player_11`) with their pitch coordinates, betting odds from ten bookmakers, and eight columns (e.g. `goal`, `card`, `corner`) whose values are XML documents listing the events of the match. 28 of its 31 foreign keys reference unique columns such as `player_api_id` instead of the primary key.

- **california_schools** describes Californian schools, with their free and reduced-price meal statistics and SAT results. It has only three tables, but they are wide (49, 29, and 11 columns), with column names such as `Percent (%) Eligible Free (K-12)`. Some rows of `schools` and `satscores` describe a district, not a school.

**Competency questions:** The questions provided by BIRD-SQL for each database are written by domain annotators, they ask what a user of the data would want to know, and, since each comes with a gold SQL query, they have a known answer on $D$. Since only the BIRD databases come with such questions, they are the datasets used in our ablation study: each is modeled with and without competency questions, by both our approach and the baseline.

For each of the five databases, we split its mini-dev questions into a *guidance set* of about 30%, stratified by difficulty, and a *held-out set* of the rest. Only the guidance set is given to the agents as competency questions (Table 2, column *CQs*). We give the agents the question text only, without the evidence and the SQL query. The held-out questions are reserved for checking whether the produced KGs can answer questions that did not influence the modeling.

**Dataset-specific handling:** The BIRD databases are SQLite files, which our converter converts directly. Primary keys, column types, and foreign keys are read from SQLite's `PRAGMA` metadata, so no further schema is needed. Beyond the general practices of Section 4.1, there is one edge case in the BIRD databases that the converter handles:

- **Foreign keys to non-key columns.** The edge of a foreign key points to the IRI of the referenced row, and that IRI is minted from the primary key. When a foreign key references another unique column, as in card_games and european_football_2, the converter first builds a lookup from the referenced value to the primary key of its row, and uses it to resolve each reference. If a value names more than one row, no edge is emitted and the literal is kept. For the same reason, foreign-key repair during sampling looks rows up by the referenced column, not by the primary key; with a primary-key lookup, it would repair almost no references in european_football_2.

### 5.1.2 Mondial

Mondial [22] is a geographical database compiled from the CIA World Factbook and other sources. It describes countries, provinces, cities, organizations, and geographical features such as rivers, lakes, mountains, islands, and deserts, and the relations between them, for example which river flows through which province, or which country is a member of which organization. With 47 tables, 74 foreign keys, and 30 tables with a composite primary key (a province, for example, is identified by its name together with its country), it is the most densely connected relational schema in our evaluation.

Mondial is distributed as PostgreSQL scripts. We convert it into SQLite first and then use our converter script. There are four specific cases that the converter handles separately for this dataset:

- **Composite column type.** Coordinates use a PostgreSQL composite type, `GeoCoord(Latitude, Longitude)`, which SQLite does not have. Each such column is split into two columns, and the tuple literals of the data are split accordingly; `CHECK` constraints on the composite are rewritten onto the new columns.

- **Missing foreign keys.** The DDL declares no foreign keys at all. Mondial documents its referential dependencies in a separate PDF. We transcribed them into the converter, which adds them as foreign-key constraints and verifies the finished database with `PRAGMA foreign_key_check`. Of the 74 foreign keys, 21 are composite.

- **Encoding errors.** Parts of the data are UTF-8 text that was once decoded as

Windows-1252 and re-encoded, such as `ÃŽle-de-France` for `Île-de-France`. These are repaired line by line before loading.

- **Tables without a primary key.** Two tables, `located` and `islandIn`, have no primary key, and their rows are identified by position.

Mondial has no competency questions with gold answers, so it is modeled without CQs only.

### 5.1.3 IMDb

The IMDb non-commercial datasets [**?** ] describe movies, series, episodes, and the people involved in them. They consist of seven tab-separated files, one per table: `title.basics`, `title.akas` (alternative titles per region and language), `title.crew`, `title.episode`, `title.principals` (the credits of a person on a title), `title.ratings`, and `name.basics`. With 207.4 million rows, IMDb is by far the largest dataset in our evaluation, although its schema is small.

IMDb is converted with `tsv2rdf.py`, which handles three properties of the data:

- **No schema.** The files contain only a header row, without types, primary keys, or foreign keys. Without a schema, every row would be identified by its position and the graph would contain no edges between rows. We therefore declare the primary keys (e.g. `tconst` for `title.basics`, and the composite key `(titleId, ordering)` for `title.akas`) and seven foreign keys inline when calling the converter. For the same reason, no literal of the IMDb generic KG is typed.

- **Null marker.** Since TSV cannot represent NULL, IMDb writes missing values as `\N`, which we declare as a null marker.

- **Multi-valued cells.** Columns such as `genres`, `primaryProfession`, and `knownForTitles` hold comma-separated lists, which are split into one literal per value with the sub-delimiter option. Lists of identifiers (`knownForTitles`, `directors`, `writers`) are split like any other column, but not linked to the rows they name. The columns `types` and `attributes` of `title.akas` separate their values with the control character `0x02` instead, which we did not split.

The sample explored by the agents takes 100 rows from each table, plus the 600 titles

and 100 persons that the sampled rows reference, which results in 1,400 rows and 10,421 triples.

**Evaluation graph:** On this sample, the analytical questions of Section 5.3.3 all have empty answers: no title of a sampled credit has a rating, no director has a person row, and the highest number of votes is 5,368. Uniform per-table sampling cannot produce connected data, since foreign-key repair only adds the parents of sampled rows, not their children. For the evaluation, we therefore built a second, connected generic KG from the complete data, $G_D^{pop}$, in exactly the layout of $G_D$: all 3,092 titles with at least 100,000 votes, with their ratings, crew, 66,452 credits, 173,143 alternative titles, and 29,701 episodes, and all 32,198 credited persons. It has 2,505,248 triples and no dangling references. The agents never saw this graph. We only execute their queries on it, which is possible since the queries were written for the same layout.

### 5.1.4 MusicBrainz

MusicBrainz [**?** ] is an open encyclopedia of music metadata: artists, releases, recordings, works, labels, places, and events, and a generic relationship model that connects any two of them, as well as the edit history of its community of editors. We use the official sample dump, restricted to 239 tables, which is the largest schema in our evaluation by an order of magnitude: 1,588 columns and 473 foreign-key columns.

The dump consists of extensionless, tab-separated PostgreSQL `COPY` files without a header; the schema is shipped separately as SQL DDL. We first add a header row to each file from the DDL, and then convert the files with `mb2rdf.py`, which reads the schema directly from the DDL. MusicBrainz declares its keys and references not as SQL constraints, but in column comments (`-- PK`, `-- references artist.id`), so these comments are the authoritative source. Parsing them required handling several details:

- **Composite primary keys.** Every column marked as `PK` belongs to the key, not only the first one. 67 tables have a composite key, for example `(recording, tag)` for `recording_tag`. Keeping only the first column would merge all rows that share it into one node, which in the dump collapsed about 4.0 million rows into 1.1 million.

- **Implicit primary keys.** MusicBrainz ships no separate primary-key definitions, and 114 of the 239 tables declare their key only through a `SERIAL` column. We infer the key from that column type; without this, every reference to these tables would be dangling. Tables with neither marker get no key.

- **DDL layout.** `CREATE TABLE` statements open their parenthesis either on the same line or on the next, and multi-line `CHECK` constraints have continuation lines that look like column definitions. Matching only the first form silently dropped 25 tables; the continuation lines produced columns that do not exist.

- **Schema-qualified names.** The Cover Art Archive tables are dumped as `cover_art_archive.art_type`, but referenced as `art_type`. The schema qualifier is removed, otherwise these files would receive no schema and all 112,142 references into them would be lost.

- **References to absent tables.** The DDL declares tables that the dump does not contain. References to them are dropped, since they could never be resolved.

The sample contains 312,695 rows of all 239 tables, with 2,683,108 triples, of which 591,333 are edges between rows. It is the only generic KG in our evaluation that cannot be inspected by reading its triples. Like Mondial and IMDb, MusicBrainz is modeled without CQs.

## 5.2 Setup

**Configurations:** We compare four configurations: our approach (GRASP) and the baseline (Section 4.4), each with and without competency questions. On the BIRD databases, we run all four. On Mondial, IMDb, and MusicBrainz, which have no competency questions, we run the two configurations without CQs. For MusicBrainz, the baseline did not produce a result: OpenCode repeatedly stalled while exploring the 2.7-million-triple file and never wrote its queries. Table 3 lists the 25 runs that we evaluate, with the run identifiers used in our experiment logs.

| Dataset | Baseline | Baseline + CQ | GRASP | GRASP + CQ |
|---|---|---|---|---|
| superhero | B6 | B7 | G19 | G20 |
| card_games | B3 | B4 | G7 | G8 |
| formula_1 | B1 | B2 | G1 | G2 |
| european_football_2 | B5 | B6 | G3 | G2 |
| california_schools | B3 | B4 | G6 | G8 |
| Mondial | B5 | – | G5 | – |
| IMDb | B2 | – | G7 | – |
| MusicBrainz | (failed) | – | G8 | – |

**Table 3:** Evaluated runs. Identifiers are the run numbers of our experiment logs and are only unique within a dataset.

All runs use the same LLM, DeepSeek v4.1 Flash, for both the generation and the review agent of GRASP and for the baseline. Each configuration was run once per dataset; we discuss the consequences of single runs in Section 5.3.2. The generic KGs are served by QLever 0.6.0.

We evaluate the output of every run on three levels, which correspond to the criteria of Section 1.2: whether the queries execute at all, how well they model the data, and how usable the result is.

**Syntactic validity:** The first level is whether $Q$ can be executed on $G_D$ and yields a valid graph. We send each CONSTRUCT query of a run to the QLever endpoint of its generic KG, request N-Triples, and write the union of the results into one file, which is the output KG $G$. We count a run as valid if every query executes and the result parses as N-Triples. We record which queries needed intervention before they could be materialized, and of what kind.

**LLM-as-a-judge:** The second level is the quality of the semantic modeling and the fidelity of $G$. Since we do not prescribe a target ontology, there is no reference graph against which $G$ could be compared triple by triple (Section 2.4). Instead, we assess the queries the way a knowledge engineer would review them, and let an LLM agent do this review, following the LLM-as-a-judge paradigm [**?** ].

The judge receives the generic KG $G_D$ (as a QLever endpoint and as a file), the queries $Q$, and the output KG $G$. It is asked to first explore $G_D$ and form its own view of how the data should be modeled, then read $Q$ in full, and then review $Q$ against nine criteria, which make the criteria of Section 1.2 concrete:

**C1. Entity and class modeling:** the right real-world things become entities, at the right granularity.

**C2. Vocabulary:** classes and predicates denote domain concepts rather than source names; each predicate has one precise meaning and a natural direction.

**C3. IRI design:** entity IRIs are minted from stable identifiers, in a domain namespace, with proper encoding.

**C4. Relationships:** the relationships of $D$ are links between entities, without $G$ being a replica of the source schema.

**C5. Self description:** every entity has an `rdf:type`, and entities with a natural name have an `rdfs:label`.

**C6. Schema coverage:** the information of every field of $D$ reaches $G$.

**C7. Soundness:** $Q$ produces only content that $D$ supports.

**C8. IRI consistency:** the same entity gets the same IRI wherever it is minted or referenced.

**C9. Literal typing:** literals carry datatypes that match their values.

C1 to C4 correspond to the semantic-modeling criteria of Section 1.2, and C6 to C9 to the fidelity criteria, with C8 covering structural integrity. C5 is added because types and labels are what makes a graph navigable for a human or an LLM. Answerability is not judged here, but evaluated with queries (see below). The criteria tell the judge what to judge, not what to look for: the examples listed under each criterion are explicitly described as a starting point, not as a checklist.

Three ground rules keep the review grounded in the data. First, every reported issue needs evidence, either a quoted fragment of $Q$ or a query on $G_D$ with its result; issues based on what the data "probably" contains are not allowed. Second, the judge may run read-only SELECT queries, but may not execute $Q$ itself. Third, it may use $G$ only to confirm or rule out a problem that it found in $Q$, for example whether a cast actually lost values, but not to judge the size or completeness of $G$ on its own. All queries are logged with a one-line summary of their result, so that every judgment can be traced.

The judge assigns each issue a severity. An issue is *major* if it loses or corrupts information, makes the graph misleading, or makes an important part of the domain wrong or hard to query; it is *minor* if the information is still present and correct, but less clear, less consistent, or less convenient than it should be. Each severity decision must be justified in one sentence. Each criterion then receives one of five levels:

The levels of the nine criteria are summed to a score out of 9, which we report scaled to 10. The judge writes its result as a JSON file, with the issues, their evidence and severity, and a rationale for each level, explaining why it chose this level and not a neighboring one. Problems that fit none of the criteria are reported separately and not

| Level | Anchor |
|---|---|
| 1 | No issues. |
| 0.75 | Only minor issues. |
| 0.5 | One major issue, or minor issues so widespread that they affect most of the graph. |
| 0.25 | Several major issues, but the criterion is still partly met. |
| 0 | The criterion is essentially not met. |

**Table 4:** Levels of the judge per criterion.

scored.

The judge is DeepSeek v4.1 Flash, run as an OpenCode agent, with one judging run per output KG. The prompt is identical for all runs, except for a configuration block that names the inputs and describes the conventions of the generic KG. Since the judge is the same model that generated the queries, it may be biased towards its own modeling choices. However, this affects both approaches equally, since both use the same model. We take the levels from the judge, but compute the totals ourselves: in two of the 25 results (superhero G19 and G20), the sum reported by the judge did not match its own levels.

**Qualitative analysis:** The third level is a manual analysis of selected runs. We read the queries and compare the modeling decisions of the two approaches on the same data. To assess usability, we write SPARQL queries for questions over each output KG, check their answers against gold SQL queries on $D$, and compare the queries by their number of triple patterns and their runtime. For the runtime measurements, each query is run five times on QLever after one warm-up run, with the result cache cleared before every run, and we report the median of QLever's total query time (computation and serialization). Since both graphs are served by the same QLever instance configuration and the queries use the same pattern order, differences in runtime reflect the modeling, for example an additional join, rather than the engine.

## 5.3 Results

### 5.3.1 Syntactic Validity

All 25 runs that produced queries yield a valid output KG. For the GRASP runs, this holds by construction: every saved query has been executed on the QLever endpoint of $G_D$ during generation, and only queries whose latest test succeeded can be submitted (Section 4.3). All thirteen GRASP query sets were therefore materialized on QLever without changes.

The baseline cannot execute its queries on a SPARQL endpoint, and six of its twelve query sets did not run on QLever as written:

- **Very wide queries.** The baseline tends to write one query per table, with one OPTIONAL per column. The queries for the widest tables, `cards` in card_games (B3, B4) and `schools` in california_schools (B3), contain up to 75 OPTIONALs in a single query. On these, QLever crashed with a stack overflow in its query planner, or did not finish within 600 seconds. GRASP never hit this problem, since a query that crashes the endpoint fails its test and has to be split by the agent.

- **Predicates built at runtime.** The lineup queries of european_football_2 (B5, B6) compute their predicate with `BIND(IRI(CONCAT(...)))` and use the variable in the predicate position, which QLever 0.6.0 rejects with an internal assertion.

- **Unsupported functions.** The IMDb baseline (B2) types its years with the cast function `xsd:gYear()`, which the engines we tried do not support: rdflib evaluates it to unbound, so the year triples silently disappear, and Oxigraph rejects the query. We replaced the cast with the equivalent `STRDT(STR(?y), xsd:gYear)`.

- **Unencoded IRIs.** The IMDb baseline mints IRIs from the values of `title.akas/types` without encoding them. Ten of these values contain the separator character `0x02` (Section 5.1), which is not allowed in an IRI. We percent-encoded these IRIs in the output.

We materialized the five query sets with the first two problems with Oxigraph instead of QLever, which executes them unchanged, and fixed the IMDb queries as described. None of these interventions changes the modeling of the queries, so the judge evaluates the queries as the baseline wrote them.

Table 5 gives the size of every output KG. The size of an output KG relative to $G_D$ says little about its quality: a graph can shrink because it drops the redundant literal copies of foreign keys, or grow because it reifies values into nodes with their own type and label. The one clear outlier is MusicBrainz, where the output has 28% of the triples of $G_D$, which already indicates that large parts of the schema are not covered (Section 5.3.2).

| Dataset | $\|G_D\|$ | Baseline | Baseline + CQ | GRASP | GRASP + CQ |
|---|---|---|---|---|---|
| superhero | 42,907 | 36,557 | 34,434 | 32,499 | 33,953 |
| card_games | 13,081 | 16,307 | 13,150 | 13,571 | 14,946 |
| formula_1 | 46,755 | 52,363 | 52,363 | 53,219 | 50,532 |
| european_football_2 | 17,369 | 31,620 | 29,711 | 33,106 | 26,866 |
| california_schools | 7,677 | 8,342 | 8,096 | 9,945 | 9,561 |
| Mondial | 26,629 | 21,976 | – | 23,249 | – |
| IMDb* | 2,505,248 | 2,780,662 | – | 2,532,669 | – |
| MusicBrainz | 2,683,108 | – | – | 751,645 | – |

**Table 5:** Number of triples of the generic KG and of each output KG. * on the evaluation graph $G_D^{pop}$.

### 5.3.2 Judge Scores

Table 6 shows the score of every run, and Table 7 the mean level of each criterion per configuration. The levels of all runs are listed in Table 8.

| Dataset | Baseline | Baseline + CQ | GRASP | GRASP + CQ |
|---|---|---|---|---|
| superhero | 8.9 (1) | **9.2** (0) | 8.9 (0) | 8.9 (0) |
| card_games | 7.2 (4) | 7.5 (3) | **7.8** (2) | 7.5 (2) |
| formula_1 | 7.2 (4) | 8.3 (3) | **9.2** (0) | 8.9 (1) |
| european_football_2 | 7.8 (3) | 7.8 (3) | **8.6** (0) | 7.8 (2) |
| california_schools | 6.7 (4) | 7.5 (3) | 8.1 (1) | **8.3** (0) |
| Mean (BIRD) | 7.6 | 8.1 | **8.5** | 8.3 |
| Mondial | 8.6 (2) | – | **8.9** (1) | – |
| IMDb | 8.6 (2) | – | **9.2** (0) | – |
| MusicBrainz | – | – | 6.4 (6) | – |

**Table 6:** Judge score per run, out of 10, with the number of major issues in parentheses. The best score per dataset is in bold.

| Configuration | C1 | C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | Score |
|---|---|---|---|---|---|---|---|---|---|---|
| Baseline | 0.65 | 0.60 | 0.90 | 0.55 | 0.75 | 0.85 | 0.80 | 0.90 | 0.80 | 7.6 |
| Baseline + CQ | 0.60 | 0.65 | 1.00 | 0.65 | 0.75 | 0.95 | 0.85 | 0.90 | 0.90 | 8.1 |
| GRASP | 0.80 | 0.75 | 0.90 | 0.80 | 0.90 | 0.85 | 0.85 | 0.90 | 0.90 | 8.5 |
| GRASP + CQ | 0.65 | 0.65 | 0.90 | 0.85 | 0.85 | 0.90 | 0.85 | 0.95 | 0.85 | 8.3 |

**Table 7:** Mean level per criterion over the five BIRD databases.

**GRASP versus baseline (RQ2):** Without competency questions, GRASP scores at least as high as the baseline on all seven datasets on which both produced a result, and higher on six of them; on superhero, both reach 8.9. The mean over the BIRD databases is 8.5 against 7.6, and the GRASP runs contain far fewer major issues:

3 against 16 on the BIRD databases, and 4 against 20 on all seven datasets. The difference is concentrated in the modeling criteria. Table 7 shows the largest gaps for relationships (C4: 0.80 against 0.55), entity and class modeling (C1: 0.80 against 0.65), vocabulary (C2: 0.75 against 0.60), and self description (C5: 0.90 against 0.75), while IRI design, coverage, and IRI consistency are on par. The judge thus finds that both approaches carry the data over equally completely, but GRASP models it better.

The major issues of the baseline fall into recurring patterns:

- **Structure kept as literals.** In european_football_2, both baseline runs keep the eight XML event columns of `Match` as one string literal per match, so goals, cards, and fouls never become entities and are never linked to their players and teams. GRASP without CQs (G3) models them.

- **Vocabulary repeating the source.** In formula_1, most predicates of both baseline runs repeat the column names (`f1:driverRef`, `f1:qualifyId`), and a single predicate `f1:position` is used for the finishing position, the qualifying position, the championship position, and the position in a lap.

- **Wrong entities.** In california_schools, both baseline runs type the ten district-level rows of `schools` as `cs:School`, and additionally mint the same districts under a second IRI, which the judge counts as a major issue under C1, C4, C7, and C8. In Mondial, the baseline models each country-specific percentage of a religion as an entity of class `Religion`, so the same religion is split into one entity per country.

- **Invented content and lost values.** In european_football_2, both baseline runs write the index of a lineup slot as `ef:shirtNumber`, although the source contains no shirt numbers. In card_games (B3), an invalid boolean cast silently drops three fields of 17 cards, and in Mondial, years are typed as `xsd:gYear` with the lexical form `1996.0`, which is not a valid year.

Some problems are shared by both approaches and point to the dataset rather than to the approach. In card_games, all four runs model each printing of a card as `mtg:Card` and never introduce the abstract card that the printings share, so the list of sets in which a card was printed is attached to every printing. This is the main reason why card_games has the lowest scores of the BIRD databases for both approaches.

The GRASP runs have few major issues, but they are of a different kind. In california_schools (G6), addresses and administrators are minted as blank nodes, which are not stable across executions. In Mondial (G5), entities that are only created as link targets, such as 261 provinces, never receive a type.

**Effect of competency questions (RQ3):** Competency questions affect the two approaches in opposite directions. For the baseline, adding CQs never lowers the score: it improves it on four of the five databases (by up to 1.1 points on formula_1) and leaves european_football_2 unchanged, and the mean rises from 7.6 to 8.1. For GRASP, adding CQs lowers the score on three databases, leaves superhero unchanged, and improves only california_schools; the mean drops from 8.5 to 8.3. With CQs, the two approaches are close: GRASP is ahead on formula_1 and california_schools, the baseline on superhero, and they are tied on card_games and european_football_2.

The criteria show where this comes from. For GRASP, the CQs lower entity modeling (C1: 0.80 to 0.65) and vocabulary (C2: 0.75 to 0.65). The clearest example is european_football_2: with CQs (G2), only two of the eight event columns become events, while the other six remain XML literals, and the event predicates `ef:player1` and `ef:player2` keep the positional source names, although their meaning changes with the type of event. We interpret this as the agent narrowing its model towards what the questions ask, despite the instruction that the questions are a minimum, not the scope of the model. For the baseline, CQs mainly improve coverage (C6: 0.85 to 0.95) and literal typing (C9: 0.80 to 0.90), that is, the questions point the agent to fields and value types that it would otherwise have missed, but they do not improve its modeling of entities.

**Schema size and complexity (RQ4):** MusicBrainz, with 239 tables, has the lowest score of all runs (6.4), with six major issues: four of them concern missing or broken relationships. The relationship query emits a relationship as soon as one of its two endpoints resolves, so only 2,723 of 31,308 relationships in $G$ have both endpoints; edits are never linked to the entities they edit; and of the nine collection-membership tables, only one is read. The vocabulary is also weaker: the endpoints of every relationship are named `mbo:entity0` and `mbo:entity1`, and one predicate `mbo:type` carries ten different meanings. Among the other datasets, the score does not simply follow the number of tables: Mondial, with 47 tables and the densest foreign-key structure, scores 8.9, while card_games, with six tables, scores lowest among the BIRD databases. What makes a dataset hard is not the number of tables as such, but how far its domain is from its schema, as with the printing/card distinction in card_games or the events hidden in XML in european_football_2, and, for MusicBrainz, the sheer number of tables that have to

be covered in one coherent model.

**Limitations:** Each configuration was run once, and each output was judged once. A difference of one level on one criterion changes the score by 0.28 points, so differences of this size between two single runs should not be over-interpreted. The consistent patterns, the GRASP lead on all seven datasets without CQs, and the opposite effect of CQs on the two approaches, are more robust than any individual difference.

| Dataset | Run | C1 | C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9 | Score |
|---|---|---|---|---|---|---|---|---|---|---|---|
| superhero | B6 | 0.5 | 0.75 | 1 | 0.75 | 1 | 1 | 1 | 1 | 1 | 8.9 |
| | B7 (CQ) | 0.75 | 1 | 1 | 0.75 | 1 | 0.75 | 1 | 1 | 1 | 9.2 |
| | G19 | 1 | 0.75 | 1 | 1 | 0.75 | 0.75 | 0.75 | 1 | 1 | 8.9 |
| | G20 (CQ) | 0.75 | 0.75 | 1 | 1 | 0.75 | 1 | 0.75 | 1 | 1 | 8.9 |
| card_games | B3 | 0.75 | 0.5 | 1 | 0.5 | 0.75 | 0.5 | 1 | 1 | 0.5 | 7.2 |
| | B4 (CQ) | 0.5 | 0.5 | 1 | 0.5 | 0.75 | 1 | 1 | 1 | 0.5 | 7.5 |
| | G7 | 0.5 | 0.75 | 1 | 0.5 | 1 | 0.75 | 0.75 | 1 | 0.75 | 7.8 |
| | G8 (CQ) | 0.5 | 0.75 | 1 | 0.5 | 0.75 | 1 | 0.75 | 1 | 0.5 | 7.5 |
| formula_1 | B1 | 1 | 0.25 | 0.75 | 0.5 | 0.5 | 1 | 1 | 1 | 0.5 | 7.2 |
| | B2 (CQ) | 0.75 | 0.25 | 1 | 1 | 0.5 | 1 | 1 | 1 | 1 | 8.3 |
| | G1 | 1 | 0.75 | 1 | 0.75 | 1 | 1 | 1 | 1 | 0.75 | 9.2 |
| | G2 (CQ) | 0.75 | 0.5 | 1 | 1 | 1 | 1 | 1 | 1 | 0.75 | 8.9 |
| european_ | B5 | 0.5 | 0.75 | 1 | 0.5 | 0.75 | 1 | 0.5 | 1 | 1 | 7.8 |
| football_2 | B6 (CQ) | 0.5 | 0.75 | 1 | 0.5 | 0.75 | 1 | 0.5 | 1 | 1 | 7.8 |
| | G3 | 0.75 | 0.75 | 1 | 0.75 | 1 | 0.75 | 0.75 | 1 | 1 | 8.6 |
| | G2 (CQ) | 0.5 | 0.5 | 0.75 | 0.75 | 0.75 | 0.75 | 1 | 1 | 1 | 7.8 |
| california_ | B3 | 0.5 | 0.75 | 0.75 | 0.5 | 0.75 | 0.75 | 0.5 | 0.5 | 1 | 6.7 |
| schools | B4 (CQ) | 0.5 | 0.75 | 1 | 0.5 | 0.75 | 1 | 0.75 | 0.5 | 1 | 7.5 |
| | G6 | 0.75 | 0.75 | 0.5 | 1 | 0.75 | 1 | 1 | 0.5 | 1 | 8.1 |
| | G8 (CQ) | 0.75 | 0.75 | 0.75 | 1 | 1 | 0.75 | 0.75 | 0.75 | 1 | 8.3 |
| Mondial | B5 | 0.5 | 0.75 | 1 | 1 | 1 | 1 | 1 | 1 | 0.5 | 8.6 |
| | G5 | 1 | 0.75 | 0.75 | 1 | 0.5 | 1 | 1 | 1 | 1 | 8.9 |
| IMDb | B2 | 0.5 | 0.5 | 1 | 1 | 0.75 | 1 | 1 | 1 | 1 | 8.6 |
| | G7 | 0.75 | 0.75 | 1 | 1 | 0.75 | 1 | 1 | 1 | 1 | 9.2 |
| MusicBrainz | G8 | 0.75 | 0.25 | 0.75 | 0.25 | 0.75 | 0.5 | 0.75 | 1 | 0.75 | 6.4 |

**Table 8:** Judge levels of all runs per criterion, and the score out of 10.

### 5.3.3 Qualitative Analysis

**IMDb** The two IMDb output KGs, B2 (baseline) and G7 (GRASP), model the same entities: titles, persons, credits, episodes, and alternative titles. Both type each title with a class per title type, link credits to their title and person, link episodes to their series, and label every entity. The judge scores them 8.6 and 9.2; the two major issues of B2 concern its `Role` entities, which are built from the credit category (`actor`, `director`, ...),

not from the role that is played. The differences that matter for querying are listed in Table 9.

| | **B2 (baseline)** | **G7 (GRASP)** |
|---|---|---|
| Rating | separate node: `?t imdb:hasRating ?r . ?r imdb:numVotes ?v` | on the title: `?t imdb:numVotes ?v` |
| Genre | node: `?t imdb:hasGenre ?g . ?g rdfs:label ?genre` | literal: `?t imdb:genre ?genre` |
| Credit category | node: `?c imdb:role role:actor` | literal: `?c imdb:category "actor"` |
| Years | `xsd:gYear` | `xsd:integer` |
| Namespace | `http://example.org/imdb/` | `http://imdb.org/` |

**Table 9:** Modeling differences between the IMDb output KGs that affect querying.

To compare their usability, we wrote nine analytical questions, each as one SPARQL query per graph, and executed them on the output KGs of the evaluation graph $G_D^{pop}$ (Section 5.1). The two queries of a question differ only where the vocabulary of the graphs differs, and they use the same pattern order. All 18 queries return the same answer as the gold SQL query on the same data. Table 10 shows the number of triple patterns of each query and its median runtime on QLever.

G7 needs fewer or equally many triple patterns on every question, 46 against 55 in total, and its queries are faster or equally fast on every question, 237 ms against 366 ms in total. Both effects trace back to two modeling decisions.

**Folding the ratings into the title.** In $D$, `title.ratings` is in a one-to-one relationship with `title.basics`, and holds only the average rating and the number of votes. B2 keeps it as a separate `Rating` node, G7 attaches both values to the title. Every question that filters, groups, or ranks by votes or rating therefore needs one more join in B2, which applies to seven of the nine questions. In Q7, which looks up the rating twice, once to find the maximum per director and once to find the movie with that maximum, B2 pays this join twice and takes twice as long:

```
# G7
?m a imdb:Movie ; imdb:director ?d ; imdb:averageRating ?r ; imdb:numVotes ?v .
# B2
?m a tt:movie ; imdb:director ?d ; imdb:hasRating ?rt .
?rt imdb:averageRating ?r ; imdb:numVotes ?v .
```

**Years as integers.** B2 types years as `xsd:gYear`, G7 as `xsd:integer`. The type of B2 is the more precise one, but SPARQL defines no arithmetic on `xsd:gYear`. In the three questions that compute an age (Q4, Q6, Q8), the natural expression `?y - ?b` silently

| # | Question | Triple patterns B2 | Triple patterns G7 | Median time (ms) B2 | Median time (ms) G7 | Speed-up |
|---|---|---|---|---|---|---|
| Q1 | All genres, with their frequency and average rating | 4 | 2 | 24 | 12 | 2.00 |
| Q2 | Persons with the most votes on average (at least 3 movies) | 6 | 5 | 38 | 35 | 1.09 |
| Q3 | Persons known for $\geq 3$ movies with $\geq 1$M votes | 5 | 4 | 14 | 12 | 1.17 |
| Q4 | Oldest directors at the time of the movie | 5* | 5 | 14 | 13 | 1.08 |
| Q5 | Series with $\geq 1$M votes, by number of episodes | 5 | 4 | 17 | 13 | 1.31 |
| Q6 | Oldest actors in movies with $\geq 100$K votes | 9* | 8 | 131 | 77 | 1.70 |
| Q7 | Highest-rated movie ($\geq 100$K votes) per director, top 10 | 12 | 10 | 95 | 47 | 2.02 |
| Q8 | Youngest directors with a movie over 1M votes | 7* | 6 | 22 | 17 | 1.29 |
| Q9 | Movies whose title contains “star” and “war” | 2 | 2 | 11 | 11 | 1.00 |
| | **Total** | **55** | **46** | **366** | **237** | **1.54** |

**Table 10:** Analytical queries on the two IMDb output KGs: number of triple patterns and median of five cold-cache runs on QLever. * additionally needs two casts.

returns no value on B2, without an error, and the query has to cast both years through a string:

```
# G7
SELECT ?name (MAX(?y - ?b) AS ?age) WHERE { ... }
# B2
SELECT ?name (MAX(xsd:integer(STR(?y)) - xsd:integer(STR(?b))) AS ?age) WHERE { ... }
```

A user who writes the natural query on B2 receives an empty result, not a wrong one, and would have to know the datatype to find the reason.

**Code lists as nodes.** B2 turns genres and credit categories into nodes with their own type and label, G7 keeps them as literals. Nodes are the more expressive choice, since further information could be attached to a genre, but none of the nine questions benefits from it. Instead, B2 needs one more hop to reach the name of a genre (Q1), and the user has to know the IRI pattern of the categories (`role:actor`) to filter by them (Q6). In Q1, the two additional hops halve the speed.

**Equal cases.** For keyword search over titles (Q9), both graphs keep the primary title as `rdfs:label`, so the queries are identical and equally fast. Neither graph defines a class hierarchy, so "series" has to be expressed as two classes (`tvSeries` and `tvMiniSeries`) in both (Q5), and "actor" as two categories (`actor` and `actress`) in both (Q6).

The absolute differences are small, at most 54 ms on a graph of 2.5 million triples, but they are systematic: G7 is never slower, and the largest speed-ups occur in the questions with the most additional hops (Q1, Q6, Q7). Together with the judge scores, this shows that the more compact modeling of G7 is not only judged better, but also makes the graph simpler and faster to query.

### Mondial

### MusicBrainz

# 6 Conclusion

# List of Figures


1 Three tables of the IMDb dataset with selected columns. . . . . 2
2 The generic KG of the IMDb tables (top) and a semantic KG of the same data (bottom). . . . . 3
3 The query “Who directed *Heat*?” over the generic KG (bottom left) and over the semantic KG (bottom right). . . . . 3
4 A CONSTRUCT query over a sample RDF KG . . . . . 13
5 GRASP sample shape format for one class . . . . . 17
6 GRASP sample structural notes . . . . . 17
7 Overview of generic-to-semantic KG pipeline . . . . . 18
8 Generation and review of CONSTRUCT queries. . . . . 23

# List of Tables


1 Rules of the direct mapping . . . . . 20
2 Overview of the datasets . . . . . 31
3 Evaluated runs. Identifiers are the run numbers of our experiment logs and are only unique within a dataset. . . . . 36
4 Levels of the judge per criterion. . . . . 39
5 Sizes of the output KGs . . . . . 41
6 Judge scores per run . . . . . 41
7 Mean judge level per criterion . . . . . 41
8 Judge levels of all runs . . . . . 44
9 Modeling differences between the IMDb output KGs that affect querying. 45
10 Analytical queries on the IMDb output KGs . . . . . 46

# Bibliography


[1] Anomaly Innovations and OpenCode contributors. OpenCode: The open source AI coding agent. https://opencode.ai, source code at https://github.com/anomalyco/opencode, 2025.

[2] Marcelo Arenas, Alexandre Bertails, Eric Prud'hommeaux, and Juan Sequeda. A direct mapping of relational data to RDF. W3c recommendation, World Wide Web Consortium (W3C), September 2012. URL https://www.w3.org/TR/2012/REC-rdb-direct-mapping-20120927/. Latest version: https://www.w3.org/TR/rdb-direct-mapping/.

[3] Julián Arenas-Guerrero, David Chaves-Fraga, Jhon Toledo, María S. Pérez, and Oscar Corcho. Morph-KGC: Scalable knowledge graph materialization with mapping partitions. *Semantic Web*, 15(1):1–20, 2024.

[4] Luigi Asprino, Enrico Daga, Aldo Gangemi, and Paul Mulholland. Knowledge graph construction with a façade: A unified method to access heterogeneous data sources on the web. *ACM Transactions on Internet Technology*, 23(1), 2023.

[5] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.

[6] Haonan Bian. Llm-empowered knowledge graph construction: A survey. *arXiv preprint arXiv:2510.20345*, 2025.

[7] Carla Castedo, Enrique Iglesias, Manuel Lama, Alberto Bugarin-Diz, Maria-Esther Vidal, and David Chaves-Fraga. Blinkg: A benchmark for llm-integrated knowledge graph generation. *arXiv preprint arXiv:2605.19518*, 2026.

[8] Richard Cyganiak. Tarql: SPARQL for tables. https://github.com/tarql/tarql. Accessed: 2026-10-09.

[9] Souripriya Das, Seema Sundara, and Richard Cyganiak. R2RML: RDB to RDF mapping language. W3C Recommendation, 2012. URL https://www.w3.org/TR/r2rml/.

[10] Anastasia Dimou, Miel Vander Sande, Pieter Colpaert, Ruben Verborgh, Erik Mannens, and Rik Van de Walle. RML: A generic language for integrated RDF mappings of heterogeneous data sources. In *Proceedings of the Workshop on Linked Data on the Web (LDOW)*, 2014.

[11] Mariano Fernández-López, Asunción Gómez-Pérez, and Natalia Juristo. METHONTOLOGY: From ontological art towards ontological engineering. In *Proceedings of the AAAI-97 Spring Symposium on Ontological Engineering*, number SS-97-06 in AAAI Technical Report, pages 33–40, Stanford, CA, USA, 1997. AAAI Press.

[12] Johannes Frey, Lars-Peter Meyer, Natanael Arndt, Felix Brei, and Kirill Bulert. Benchmarking the abilities of large language models for rdf knowledge graph creation and comprehension: how well do llms speak turtle? *arXiv preprint arXiv:2309.17122*, 2023.

[13] Michael Grüninger and Mark S. Fox. Methodology for the design and evaluation of ontologies. In *Proceedings of the IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.

[14] Ali Hariri, Stéphane Jean, and Mickaël Baron. Towards automating rdf extraction for archaeological knowledge graphs with llms. In *International Conference on Database and Expert Systems Applications*, pages 83–97. Springer, 2025.

[15] Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C Recommendation, March 2013. URL `https://www.w3.org/TR/sparql11-query/`.

[16] Marvin Hofer, Johannes Frey, and Erhard Rahm. Towards self-configuring knowledge graph construction pipelines using llms-a case study with rml. In *Fifth International Workshop on Knowledge Graph Construction@ ESWC2024*, 2024.

[17] Vamsi Krishna Kommineni, Birgitta König-Ries, and Sheeba Samuel. From human experts to machines: An LLM supported approach to ontology and knowledge graph construction. *arXiv preprint arXiv:2403.08345*, 2024.

[18] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded

text-to-SQLs. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2305.03111.

[19] Anna Sofia Lippolis, Miguel Ceriani, Sara Zuppiroli, and Andrea Giovanni Nuzzolese. Ontogenia: Ontology generation with metacognitive prompting in large language models. In *The Semantic Web: ESWC 2024 Satellite Events*, 2024.

[20] Anna Sofia Lippolis, Mohammad Javad Saeedizade, Robin Keskisärkkä, Sara Zuppiroli, Miguel Ceriani, Aldo Gangemi, Eva Blomqvist, and Andrea Giovanni Nuzzolese. Ontology generation using large language models, March 2025. URL https://arxiv.org/abs/2503.05388.

[21] Ioana Ramona Martin, Tudor Cioara, Ionut Anghel, and Gabriel Arcas. Performance evaluation of large language models for automated knowledge graph generation. *Machine Learning with Applications*, page 100923, 2026.

[22] Wolfgang May. Information extraction and integration with FLORID: The MONDIAL case study. Technical Report 131, Universität Freiburg, Institut für Informatik, 1999. Available from http://dbis.informatik.uni-goettingen.de/Mondial.

[23] Lars-Peter Meyer, Johannes Frey, Desiree Heim, Felix Brei, Claus Stadler, Kurt Junghanns, and Michael Martin. Llm-kg-bench 3.0: A compass for semantic technology capabilities in the ocean of llms. In *European Semantic Web Conference*, pages 280–296. Springer, 2025.

[24] Maria Angela Pellegrino and Gabriele Tuozzo. From quality reports to knowledge graphs: a case study on csv-to-kg transformation. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 1626–1632, 2025.

[25] María Poveda-Villalón, Alba Fernández-Izquierdo, Mariano Fernández-López, and Raúl García-Castro. LOT: An industrial oriented ontology engineering framework. *Engineering Applications of Artificial Intelligence*, 111:104755, 2022.

[26] Alex Randles and Declan O'Sullivan. R2 [rml]-chatgpt framework. In *Fifth International Workshop on Knowledge Graph Construction@ ESWC2024*, 2024.

[27] Mohammad Javad Saeedizade and Eva Blomqvist. Navigating ontology development with large language models. In Albert Meroño Peñuela, Anastasia Dimou, Raphaël Troncy, Olaf Hartig, Maribel Acosta, Mehwish Alam, Heiko Paulheim, and Pasquale Lisena, editors, *The Semantic Web: 21st International Conference, ESWC 2024, Proceedings, Part I*, volume 14664 of *Lecture Notes in Computer Science*, pages 143–161, Cham, 2024. Springer. doi: 10.1007/978-3-031-60626-7_8.

[28] Wilma Johanna Schmidt, Irlán Grangel-González, Adrian Paschke, and Evgeny Kharlamov. LLM-supported manufacturing mapping generation. *Transactions on Graph Data and Knowledge*, 3(3):5:1–5:22, 2025. doi: 10.4230/TGDK.3.3.5.

[29] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. A multi-agent system for semantic mapping of relational data to knowledge graphs. *arXiv preprint arXiv:2511.06455*, 2025.

[30] Gabriele Tuozzo et al. Moving from tabular knowledge graph quality assessment to rdf triples leveraging chatgpt. In *TEXT2KG/DQMLKG@ ESWC*, page 9, 2024.

[31] Mikel Val-Calvo, Mikel Egaña Aranguren, Juan Mulero-Hernández, Ginés Almagro-Hernández, Prashant Deshmukh, José Antonio Bernabé-Díaz, Paola Espinoza-Arias, José Luis Sánchez-Fernández, Juergen Mueller, and Jesualdo Tomás Fernández-Breis. OntoGenix: Leveraging large language models for enhanced ontology engineering from datasets. *Information Processing & Management*, 62 (3), 2025.

[32] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57, 1985.

[33] Sebastian Walter and Hannah Bast. GRASP: Generic reasoning and SPARQL generation across knowledge graphs. In *The Semantic Web – ISWC 2025: 24th International Semantic Web Conference, Proceedings, Part I*, volume 16140 of *Lecture Notes in Computer Science*, pages 271–289, Cham, 2025. Springer.

[34] Guohui Xiao, Lin Ren, Guilin Qi, Haohan Xue, Marco Di Panfilo, and Davide Lanti. LLM4VKG: Leveraging large language models for virtual knowledge graph construction. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, pages 4715–4723, 2025. doi: 10.24963/ijcai.2025/525.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.